

Hướng dẫn chi tiết C# ASP.NET từ cơ bản đến nâng cao

Giới thiệu

Chào mừng bạn đến với tài liệu hướng dẫn toàn diện về C# ASP.NET. Tài liệu này được thiết kế để cung cấp cho bạn kiến thức từ cơ bản đến nâng cao, giúp bạn trở thành một nhà phát triển ASP.NET thành thạo. Chúng ta sẽ đi sâu vào các khái niệm, kiến trúc, và thực hành tốt nhất, kèm theo các ví dụ code chi tiết để bạn có thể áp dụng ngay lập tức.

Mục lục

1. Junior Level: Nền tảng cơ bản

- Giới thiệu về .NET và ASP.NET Core
- Cú pháp C# cơ bản
- Cấu trúc dự án ASP.NET Core
- Middleware và Request Pipeline
- Dependency Injection
- Razor Pages và MVC cơ bản
- Làm việc với Static Files
- Xử lý lỗi cơ bản

2. Middle Level: Phát triển ứng dụng nâng cao

- Làm việc với dữ liệu (Entity Framework Core)
- CRUD Operations
- Validation và Model Binding
- Authentication và Authorization
- API Development (RESTful APIs)
- Logging và Monitoring
- Unit Testing

3. Senior Level: Tối ưu hóa và bảo mật

- Performance Optimization (Caching, Asynchronous Programming)
- Security Best Practices (XSS, CSRF, SQL Injection)
- Advanced Authentication and Authorization (IdentityServer, JWT)
- Microservices Architecture (Basic Concepts)
- Containerization (Docker)
- Deployment Strategies (Azure, AWS)
- Integration Testing
- Real-time Communication (SignalR)
- Advanced Entity Framework Core

4. Principal Level: Kiến trúc hệ thống và chiến lược

- Distributed Systems Patterns (Event-Driven Architecture, Saga Pattern, Idempotency)
- Advanced Performance Tuning and Scalability
- Domain-Driven Design (DDD) và Clean Architecture
- Observability (Distributed Tracing, Metrics, Logging)
- Advanced Security Topics (Threat Modeling, Security Headers, OWASP Top 10)
- Advanced Deployment and DevOps

1. Junior Level: Nền tảng cơ bản

Cấp độ này sẽ trang bị cho bạn những kiến thức cơ bản nhất để bắt đầu hành trình phát triển web với ASP.NET. Bạn sẽ học cách thiết lập môi trường, hiểu cấu trúc dự án, và xây dựng các ứng dụng web đơn giản.

1.1 Giới thiệu về .NET và ASP.NET Core

ASP.NET là một framework mã nguồn mở, đa nền tảng được phát triển bởi Microsoft, dùng để xây dựng các ứng dụng web hiện đại, mạnh mẽ và có khả năng mở rộng. Nó là một phần của hệ sinh thái .NET, một nền tảng phát triển phần mềm rộng lớn hỗ trợ nhiều ngôn ngữ lập trình, trong đó C# là ngôn ngữ chính.

Sự khác biệt giữa .NET Framework và .NET Core (nay là .NET)

Trước đây, Microsoft có hai nền tảng chính: .NET Framework và .NET Core. Hiện tại, .NET Core đã phát triển thành .NET (từ phiên bản .NET 5 trở đi), thống nhất các tính năng và mang lại nhiều lợi ích vượt trội:

- **.NET Framework:** Là phiên bản cũ hơn, chỉ chạy trên Windows. Nó bao gồm các công nghệ như ASP.NET Web Forms, ASP.NET MVC 5, WPF, và Windows Forms. .NET Framework vẫn được hỗ trợ nhưng không còn được phát triển tính năng mới.
- **.NET (trước đây là .NET Core):** Là phiên bản hiện đại, đa nền tảng (Windows, Linux, macOS), mã nguồn mở và hiệu suất cao. Nó được thiết kế để xây dựng các ứng dụng web, dịch vụ microservices, ứng dụng di động, và ứng dụng desktop hiện đại. ASP.NET Core là một phần của .NET, tập trung vào phát triển web.

Tại sao nên học ASP.NET Core?

1. **Đa nền tảng:** Phát triển và triển khai ứng dụng trên Windows, Linux, và macOS.
2. **Hiệu suất cao:** Được tối ưu hóa để đạt hiệu suất vượt trội so với các phiên bản ASP.NET cũ.
3. **Mã nguồn mở:** Cộng đồng lớn mạnh và minh bạch trong quá trình phát triển.
4. **Linh hoạt:** Hỗ trợ nhiều mô hình phát triển như MVC, Razor Pages, và Web API.
5. **Tích hợp dễ dàng:** Tích hợp chặt chẽ với các công nghệ frontend hiện đại như React, Angular, Vue.js.
6. **Cloud-ready:** Dễ dàng triển khai lên các nền tảng đám mây như Azure, AWS, Google Cloud.

Trong tài liệu này, chúng ta sẽ tập trung vào **ASP.NET Core** vì đây là tương lai của phát triển web với .NET.

1.2 Cú pháp C# cơ bản

C# là ngôn ngữ lập trình chính được sử dụng trong ASP.NET Core. Để làm việc hiệu quả với ASP.NET Core, bạn cần nắm vững các khái niệm cơ bản của C#.

1.2.1 Biến và Kiểu dữ liệu

C# là một ngôn ngữ kiểu tĩnh (statically-typed), nghĩa là bạn phải khai báo kiểu dữ liệu của biến trước khi sử dụng. Các kiểu dữ liệu cơ bản bao gồm:

- **Số nguyên:** `int`, `long`, `short`, `byte`

- **Số thực:** `float`, `double`, `decimal`
- **Ký tự:** `char`
- **Chuỗi:** `string`
- **Boolean:** `bool`

```
// Khai báo và khởi tạo biến
int age = 30;
string name = "John Doe";
bool isActive = true;
double price = 19.99;

// Sử dụng từ khóa 'var' để trình biên dịch tự suy luận kiểu
var quantity = 100;
var message = "Hello, C#!";
```

1.2.2 Toán tử

C# hỗ trợ các loại toán tử phổ biến:

- **Toán tử số học:** `+`, `-`, `*`, `/`, `%`
- **Toán tử so sánh:** `==`, `!=`, `<`, `>`, `<=`, `>=`
- **Toán tử logic:** `&&` (AND), `||` (OR), `!` (NOT)
- **Toán tử gán:** `=`, `+=`, `-=`, `*=` v.v.

```
int a = 10, b = 5;
int sum = a + b; // 15
bool isEqual = (a == b); // false
bool condition = (a > 0 && b < 10); // true
```

1.2.3 Cấu trúc điều khiển

- **if-else:** Thực thi code dựa trên điều kiện.

```

if (age >= 18)
{
    Console.WriteLine("Bạn đủ tuổi bầu cử.");
}
else
{
    Console.WriteLine("Bạn chưa đủ tuổi bầu cử.");
}

```

- **switch**: Lựa chọn thực thi code dựa trên giá trị của một biến.

```

string day = "Monday";
switch (day)
{
    case "Monday":
        Console.WriteLine("Hôm nay là thứ Hai.");
        break;
    case "Friday":
        Console.WriteLine("Hôm nay là thứ Sáu.");
        break;
    default:
        Console.WriteLine("Một ngày khác trong tuần.");
        break;
}

```

1.2.4 Vòng lặp

- **for**: Lặp lại code một số lần nhất định.

```

for (int i = 0; i < 5; i++)
{
    Console.WriteLine($"Lần lặp thứ {i}");
}

```

- **foreach**: Lặp qua các phần tử trong một tập hợp.

```

string[] names = { "Alice", "Bob", "Charlie" };
foreach (string n in names)
{
    Console.WriteLine($"Tên: {n}");
}

```

- **while**: Lặp lại code cho đến khi điều kiện sai.

```
int count = 0;
while (count < 3)
{
    Console.WriteLine($"Count: {count}");
    count++;
}
```

1.2.5 Phương thức (Methods)

Phương thức là một khối code thực hiện một tác vụ cụ thể. Chúng giúp tổ chức code và tái sử dụng.

```
public class Calculator
{
    public int Add(int num1, int num2)
    {
        return num1 + num2;
    }

    public void Greet(string personName)
    {
        Console.WriteLine($"Xin chào, {personName}!");
    }
}

// Sử dụng phương thức
Calculator calc = new Calculator();
int result = calc.Add(5, 3); // result = 8
calc.Greet("World"); // In ra "Xin chào, World!"
```

1.2.6 Lớp và Đối tượng (Classes and Objects)

C# là một ngôn ngữ hướng đối tượng (Object-Oriented Programming - OOP). Lớp là bản thiết kế (blueprint) để tạo ra các đối tượng. Đối tượng là một thể hiện (instance) của một lớp.

```
public class Car
{
```

```

// Thuộc tính (Properties)
public string Make { get; set; }
public string Model { get; set; }
public int Year { get; set; }

// Phương thức khởi tạo (Constructor)
public Car(string make, string model, int year)
{
    Make = make;
    Model = model;
    Year = year;
}

// Phương thức (Method)
public void DisplayInfo()
{
    Console.WriteLine($"Xe: {Year} {Make} {Model}");
}
}

// Tạo đối tượng từ lớp Car
Car myCar = new Car("Toyota", "Camry", 2022);
myCar.DisplayInfo(); // In ra "Xe: 2022 Toyota Camry"

```

1.2.7 Namespace

Namespace được sử dụng để tổ chức code và tránh xung đột tên. Bạn sử dụng từ khóa `using` để import các namespace cần thiết.

```

using System; // Import namespace System
using System.Collections.Generic;

namespace MyFirstApp
{
    public class Program
    {
        public static void Main(string[] args)
        {
            Console.WriteLine("Hello from Namespace!");
        }
    }
}

```

```
        List<string> names = new List<string>();  
    }  
}  
}
```

Đây là những kiến thức C# cơ bản nhất mà bạn cần nắm vững trước khi đi sâu vào ASP.NET Core. Nếu bạn chưa quen với C#, hãy dành thời gian thực hành các ví dụ này để làm quen với cú pháp và các khái niệm.

1.3. Cấu trúc dự án ASP.NET Core

Khi bạn tạo một dự án ASP.NET Core mới, Visual Studio hoặc .NET CLI sẽ tạo ra một cấu trúc thư mục và tệp tin tiêu chuẩn. Việc hiểu cấu trúc này là rất quan trọng để bạn có thể tổ chức mã nguồn một cách hiệu quả và dễ dàng bảo trì.

Tạo một dự án ASP.NET Core mới

Bạn có thể tạo dự án bằng Visual Studio hoặc .NET CLI.

Sử dụng .NET CLI:

```
dotnet new webapp -n MyFirstWebApp  
cd MyFirstWebApp
```

Lệnh `dotnet new webapp` tạo một dự án ASP.NET Core mới sử dụng mẫu Razor Pages. Nếu bạn muốn tạo một dự án MVC, bạn có thể dùng `dotnet new mvc`.

Cấu trúc thư mục cơ bản:

Sau khi tạo, cấu trúc dự án sẽ trông tương tự như sau:

```
MyFirstWebApp/  
├─ Properties/  
│   └─ launchSettings.json  
├─ wwwroot/  
│   └─ css/
```



```
|   └─ js/
|   └─ lib/
└─ Pages/ (hoặc Controllers/ và Views/ nếu là dự án MVC)
|   └─ Shared/
|       └─ _Layout.cshtml
|   └─ _ViewImports.cshtml
|   └─ _ViewStart.cshtml
|   └─ Index.cshtml
|   └─ Error.cshtml
└─ appsettings.json
└─ appsettings.Development.json
└─ Program.cs
└─ MyFirstWebApp.csproj
└─ README.md
```

Giải thích các tệp và thư mục chính:

- **MyFirstWebApp.csproj**: Đây là tệp dự án C# (C# Project file). Nó chứa các cài đặt dự án, tham chiếu đến các gói NuGet, thông tin về framework mục tiêu và các tệp nguồn được bao gồm trong dự án. Khi bạn thêm các gói NuGet hoặc thay đổi cài đặt dự án, tệp này sẽ được cập nhật.

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

</Project>
```

- **Program.cs**: Đây là tệp khởi động chính của ứng dụng. Nó chứa phương thức **Main** là điểm vào của ứng dụng. Trong tệp này, bạn sẽ cấu hình các dịch vụ (services) cần thiết cho ứng dụng (như Dependency Injection, Entity Framework, Authentication) và định nghĩa pipeline xử lý yêu cầu HTTP (request pipeline) bằng cách thêm các middleware.

```
var builder = WebApplication.CreateBuilder(args);
```

```

// Add services to the container.
builder.Services.AddRazorPages(); // Hoặc
AddControllersWithViews() cho MVC

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    // The default HSTS value is 30 days. You may want to
    change this for production scenarios, see
    https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();

app.UseRouting();

app.UseAuthorization();

app.MapRazorPages(); // Hoặc MapControllerRoute() cho MVC

app.Run();

```

- **appsettings.json**: Tập cấu hình ứng dụng. Nó chứa các cài đặt như chuỗi kết nối cơ sở dữ liệu, khóa API, và các cài đặt khác mà ứng dụng cần. Bạn có thể có các tập cấu hình riêng cho từng môi trường (ví dụ: `appsettings.Development.json`, `appsettings.Production.json`).

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

- **Properties/launchSettings.json**: Cấu hình cách ứng dụng được khởi chạy trong các môi trường phát triển khác nhau (ví dụ: cấu hình cổng, biến môi trường). Tập này chỉ được sử dụng trong quá trình phát triển.
- **wwwroot/**: Thư mục này chứa các tệp tĩnh (static files) mà trình duyệt có thể truy cập trực tiếp, như CSS, JavaScript, hình ảnh, và thư viện client-side (ví dụ: Bootstrap, jQuery). Các tệp trong thư mục này được phục vụ trực tiếp bởi web server.
- **Pages/ (cho Razor Pages)**:
 - Chứa các tệp **.cshtml** là các trang Razor Pages. Mỗi tệp **.cshtml** thường có một tệp code-behind **.cshtml.cs** tương ứng chứa logic xử lý.
 - **_Layout.cshtml**: Tệp layout chính, định nghĩa cấu trúc HTML chung cho tất cả các trang (header, footer, navigation).
 - **_ViewImports.cshtml**: Chứa các directive **using** và các helper được áp dụng cho tất cả các trang trong thư mục và các thư mục con.
 - **_ViewStart.cshtml**: Được thực thi trước mỗi trang Razor Pages và thường được dùng để chỉ định tệp layout mặc định.
- **Controllers/ và Views/ (cho MVC)**:
 - **Controllers/**: Chứa các lớp controller xử lý yêu cầu HTTP, chứa logic nghiệp vụ và trả về các View hoặc dữ liệu.
 - **Views/**: Chứa các tệp **.cshtml** là các View, chịu trách nhiệm hiển thị giao diện người dùng. Thường có các thư mục con tương ứng với tên controller (ví dụ: **Views/Home/Index.cshtml**).
 - **Views/Shared/**: Chứa các View được chia sẻ giữa các controller, ví dụ **_Layout.cshtml**.

Việc hiểu rõ vai trò của từng thành phần trong cấu trúc dự án sẽ giúp bạn tổ chức mã nguồn một cách hiệu quả và dễ dàng bảo trì. Trong các phần tiếp theo, chúng ta sẽ đi sâu vào từng thành phần và cách chúng hoạt động trong ASP.NET Core.

1.4 Middleware và Request Pipeline

Trong ASP.NET Core, mọi yêu cầu HTTP (request) đều được xử lý thông qua một chuỗi các thành phần gọi là **middleware**. Các middleware này được sắp xếp theo một pipeline (đường ống), và mỗi middleware có thể thực hiện một tác vụ cụ thể, sau đó chuyển yêu cầu cho middleware tiếp theo trong pipeline, hoặc kết thúc xử lý yêu cầu và trả về phản hồi (response).

Khái niệm Middleware:

Middleware là một phần mềm được lắp ráp vào pipeline xử lý yêu cầu của ứng dụng để xử lý các yêu cầu và phản hồi. Mỗi thành phần middleware:

- Chọn có chuyển yêu cầu đến thành phần tiếp theo trong pipeline hay không.
- Có thể thực hiện công việc trước và sau khi thành phần tiếp theo trong pipeline được gọi.

Request Pipeline:

Request pipeline được cấu hình trong phương thức `Configure` của lớp `Startup.cs` (hoặc trực tiếp trong `Program.cs` đối với .NET 6+). Thứ tự các middleware được thêm vào pipeline là rất quan trọng, vì chúng được thực thi theo thứ tự đó.

Ví dụ về cấu hình Middleware trong `Program.cs` (.NET 6+):

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
```

```

    // The default HSTS value is 30 days. You may want to change
    this for production scenarios, see https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}

app.UseHttpsRedirection(); // Chuyển hướng HTTP sang HTTPS
app.UseStaticFiles();      // Phục vụ các tệp tĩnh (HTML, CSS, JS,
hình ảnh)

app.UseRouting();          // Định tuyến các yêu cầu đến các
endpoint phù hợp

app.UseAuthorization();     // Xử lý ủy quyền (kiểm tra quyền truy
cập)

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();

```

Giải thích các Middleware phổ biến:

- **app.UseExceptionHandler()**: Bắt các ngoại lệ (exceptions) trong pipeline và hiển thị trang lỗi thân thiện.
- **app.UseHsts()**: Thêm HTTP Strict Transport Security Protocol (HSTS) header, buộc trình duyệt chỉ sử dụng HTTPS.
- **app.UseHttpsRedirection()**: Chuyển hướng các yêu cầu HTTP không an toàn sang HTTPS.
- **app.UseStaticFiles()**: Cho phép ứng dụng phục vụ các tệp tĩnh từ thư mục `wwwroot` (ví dụ: CSS, JavaScript, hình ảnh).
- **app.UseRouting()**: Thêm khả năng định tuyến vào pipeline, khớp các URL đến các endpoint (ví dụ: Controller action, Razor Page).
- **app.UseAuthorization()**: Thực hiện kiểm tra ủy quyền, đảm bảo người dùng có quyền truy cập vào tài nguyên được yêu cầu.

- `app.MapControllerRoute()` / `app.MapRazorPages()`: Định nghĩa các tuyến đường (routes) cho MVC Controller hoặc Razor Pages. Đây là nơi các yêu cầu được ánh xạ tới các xử lý cụ thể.

Tạo một Middleware tùy chỉnh đơn giản:

Bạn có thể tạo middleware của riêng mình. Một middleware đơn giản có thể được viết dưới dạng một phương thức mở rộng (extension method) cho `IApplicationBuilder`.

```
// Trong một tệp riêng, ví dụ: CustomMiddleware.cs
using Microsoft.AspNetCore.Builder;
using Microsoft.AspNetCore.Http;
using System.Threading.Tasks;

public class MyCustomMiddleware
{
    private readonly RequestDelegate _next;

    public MyCustomMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        // Logic trước khi chuyển yêu cầu cho middleware tiếp theo
        Console.WriteLine($"Request đến: {context.Request.Path}");

        await _next(context); // Chuyển yêu cầu cho middleware tiếp theo

        // Logic sau khi middleware tiếp theo đã xử lý yêu cầu
        Console.WriteLine($"Response trạng thái: {context.Response.StatusCode}");
    }
}

public static class MyCustomMiddlewareExtensions
{
    public static IApplicationBuilder UseMyCustomMiddleware(
```

```
        this IApplicationBuilder builder)
    {
        return builder.UseMiddleware<MyCustomMiddleware>();
    }
}
```

Để sử dụng middleware này, bạn thêm nó vào `Program.cs` (hoặc `Startup.cs`):

```
// Trong Program.cs, sau app.UseRouting() và trước
app.UseAuthorization()
app.UseMyCustomMiddleware();
```

Việc hiểu rõ cách hoạt động của middleware và request pipeline là rất quan trọng để cấu hình và mở rộng ứng dụng ASP.NET Core của bạn một cách hiệu quả.

1.5 Dependency Injection

Dependency Injection (DI) là một kỹ thuật thiết kế phần mềm được sử dụng rộng rãi trong ASP.NET Core để đạt được sự tách biệt giữa các thành phần (separation of concerns) và làm cho code dễ kiểm thử, dễ bảo trì hơn. Thay vì một đối tượng tự tạo ra các đối tượng mà nó phụ thuộc (dependencies), các đối tượng này sẽ được cung cấp (injected) từ bên ngoài.

Tại sao cần Dependency Injection?

- **Giảm sự phụ thuộc chặt chẽ (tight coupling):** Các lớp không cần biết cách tạo ra các dependency của chúng, chỉ cần biết cách sử dụng chúng. Điều này giúp thay đổi một dependency mà không ảnh hưởng đến lớp sử dụng.
- **Tăng khả năng kiểm thử (testability):** Dễ dàng thay thế các dependency bằng các đối tượng giả (mock objects) trong quá trình kiểm thử đơn vị (unit testing).
- **Tăng khả năng tái sử dụng (reusability):** Các thành phần có thể được sử dụng lại trong nhiều ngữ cảnh khác nhau.
- **Dễ bảo trì (maintainability):** Code trở nên dễ hiểu và dễ quản lý hơn.

Cách hoạt động của DI trong ASP.NET Core:

ASP.NET Core có một container DI tích hợp sẵn, được gọi là **Service Provider**. Bạn đăng ký các dịch vụ (services) vào container này, và sau đó container sẽ tự động cung cấp các dịch vụ đó cho các lớp cần chúng thông qua constructor injection.

Các loại Lifetime của Service:

Khi đăng ký một dịch vụ, bạn cần chỉ định lifetime của nó, tức là cách thức và thời điểm container tạo ra và hủy bỏ instance của dịch vụ đó:

1. **Transient**: Một instance mới của dịch vụ được tạo ra mỗi khi nó được yêu cầu. Phù hợp cho các dịch vụ nhẹ, không trạng thái (stateless) hoặc các dịch vụ cần một instance riêng biệt cho mỗi thao tác.

```
services.AddTransient<IMyService, MyService>();
```

2. **Scoped**: Một instance của dịch vụ được tạo ra một lần cho mỗi request HTTP. Instance này được chia sẻ trong suốt vòng đời của request đó. Phù hợp cho các dịch vụ có trạng thái (stateful) trong phạm vi một request, ví dụ như `DbContext` của Entity Framework Core.

```
services.AddScoped<IMyService, MyService>();
```

3. **Singleton**: Một instance duy nhất của dịch vụ được tạo ra lần đầu tiên nó được yêu cầu và được sử dụng lại cho tất cả các yêu cầu tiếp theo trong suốt vòng đời của ứng dụng. Phù hợp cho các dịch vụ không trạng thái, hoặc các dịch vụ có chi phí khởi tạo cao và cần được chia sẻ toàn cục.

```
services.AddSingleton<IMyService, MyService>();
```

Ví dụ về sử dụng Dependency Injection:

Giả sử chúng ta có một interface `ILogger` và một lớp `ConsoleLogger` triển khai interface này:

```
// 1. Định nghĩa Interface
public interface ILogger
{
    void Log(string message);
}
```



```
// 2. Triển khai Interface
public class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"[ConsoleLogger] {message}");
    }
}

// 3. Một lớp cần sử dụng ILogger
public class HomeController : Controller
{
    private readonly ILogger _logger;

    // Constructor Injection: ILogger được inject vào đây
    public HomeController(ILogger logger)
    {
        _logger = logger;
    }

    public IActionResult Index()
    {
        _logger.Log("Truy cập trang Index của Home Controller.");
        return View();
    }
}
```

Đăng ký dịch vụ trong Program.cs (.NET 6+):

Để ASP.NET Core biết cách cung cấp `ILogger` khi `HomeController` yêu cầu, bạn cần đăng ký nó trong `Program.cs`:

```
var builder = WebApplication.CreateBuilder(args);

// Đăng ký dịch vụ ILogger và triển khai ConsoleLogger với lifetime
là Singleton
builder.Services.AddSingleton<ILogger, ConsoleLogger>();

// Add services to the container.
builder.Services.AddControllersWithViews();

var app = builder.Build();

// ... (cấu hình middleware)

app.Run();
```

Khi ứng dụng chạy, ASP.NET Core sẽ tự động tạo một instance của `ConsoleLogger` (hoặc bất kỳ triển khai `ILogger` nào bạn đã đăng ký) và truyền nó vào constructor của `HomeController` khi `HomeController` được tạo ra. Điều này giúp `HomeController` không cần biết gì về cách `ILogger` được tạo ra, chỉ cần biết cách sử dụng nó.

Dependency Injection là một khái niệm cốt lõi trong ASP.NET Core và là một kỹ thuật quan trọng để xây dựng các ứng dụng có cấu trúc tốt và dễ bảo trì.

1.6 Razor Pages và MVC cơ bản

ASP.NET Core cung cấp hai mô hình chính để xây dựng ứng dụng web có giao diện người dùng: **Razor Pages** và **Model-View-Controller (MVC)**. Cả hai đều sử dụng Razor Syntax để tạo View, nhưng cách tổ chức code và xử lý yêu cầu có sự khác biệt.

1.6.1 Razor Pages

Razor Pages là một mô hình phát triển web dựa trên trang (page-based) được giới thiệu trong ASP.NET Core 2.0. Nó đơn giản hóa việc phát triển các ứng dụng web nhỏ hơn hoặc các phần của ứng dụng lớn hơn bằng cách kết hợp logic xử lý yêu cầu và UI trong một tệp duy nhất (hoặc hai tệp liên quan: `.cshtml` và `.cshtml.cs`).

Đặc điểm chính:

- **Page-based:** Mỗi trang Razor Page là một tệp `.cshtml` và có thể có một tệp code-behind `.cshtml.cs` tương ứng.
- **Tự chứa (Self-contained):** Logic xử lý yêu cầu (handler methods) và dữ liệu của trang được đặt gần với markup HTML, giúp dễ dàng quản lý các trang riêng lẻ.
- **Đơn giản hóa:** Giảm bớt sự phức tạp của mô hình MVC truyền thống cho các tác vụ đơn giản.

Cấu trúc thư mục Razor Pages:

Razor Pages thường nằm trong thư mục `Pages` ở thư mục gốc của dự án.

```
MyRazorApp/  
├── Pages/  
│   ├── Index.cshtml  
│   ├── Index.cshtml.cs  
│   ├── Privacy.cshtml  
│   ├── Privacy.cshtml.cs  
│   └── Shared/  
│       ├── _Layout.cshtml  
│       └── _ValidationScriptsPartial.cshtml  
├── Program.cs  
├── appsettings.json  
└── ...
```

Ví dụ Razor Page đơn giản:

`Pages/Index.cshtml`:

```
@page  
@model IndexModel  
@{  
    ViewData["Title"] = "Trang chủ";  
}  
  
<div class="text-center">  
    <h1 class="display-4">Chào mừng</h1>
```

```
<p>Học về <a href="https://docs.microsoft.com/aspnet/core">xây  
dựng ứng dụng Web với ASP.NET Core</a>.</p>  
<p>Thông báo từ Model: @Model.Message</p>  
<form method="post">  
    <input type="text" asp-for="InputMessage" />  
    <button type="submit">Gửi</button>  
</form>  
</div>
```

Pages/Index.cshtml.cs:

```
using Microsoft.AspNetCore.Mvc;  
using Microsoft.AspNetCore.Mvc.RazorPages;  
  
namespace MyRazorApp.Pages  
{  
    public class IndexModel : PageModel  
    {  
        public string Message { get; set; } = "Đây là thông báo mặc  
định.";   
  
        [BindProperty]  
        public string InputMessage { get; set; }  
  
        public void OnGet()  
        {  
            // Logic khi trang được tải lần đầu (GET request)  
            Message = "Chào mừng bạn đến với Razor Pages!";  
        }  
  
        public IActionResult OnPost()  
        {  
            // Logic khi form được gửi (POST request)  
            if (!string.IsNullOrEmpty(InputMessage))  
            {  
                Message = $"Bạn đã gửi: {InputMessage}";  
            }  
            else  
            {  

```

```

        Message = "Bạn chưa nhập gì cả!";
    }
    return Page(); // Trả về cùng trang
}
}
}

```

Trong `Program.cs`, bạn cần thêm `app.MapRazorPages()` để kích hoạt Razor Pages:

```

// ...
app.UseRouting();
app.UseAuthorization();

app.MapRazorPages(); // Kích hoạt Razor Pages

app.Run();

```

1.6.2 Model-View-Controller (MVC)

MVC là một kiến trúc thiết kế phần mềm chia ứng dụng thành ba thành phần chính:

- **Model:** Đại diện cho dữ liệu và logic nghiệp vụ của ứng dụng. Nó không quan tâm đến giao diện người dùng.
- **View:** Chịu trách nhiệm hiển thị giao diện người dùng. Nó nhận dữ liệu từ Controller và hiển thị chúng.
- **Controller:** Xử lý các yêu cầu của người dùng, tương tác với Model để lấy hoặc cập nhật dữ liệu, và sau đó chọn View phù hợp để hiển thị kết quả.

Đặc điểm chính:

- **Tách biệt rõ ràng:** Phân chia trách nhiệm rõ ràng giữa dữ liệu, logic và giao diện.
- **Kiểm thử dễ dàng:** Do sự tách biệt, các thành phần có thể được kiểm thử độc lập.
- **Phù hợp cho ứng dụng lớn:** Thích hợp cho các ứng dụng phức tạp với nhiều logic nghiệp vụ và giao diện người dùng đa dạng.

Cấu trúc thư mục MVC:

Các thành phần MVC thường được tổ chức trong các thư mục riêng biệt: `Controllers`, `Models`, và `Views`.

```
MyMvcApp/  
├─ Controllers/  
│   └─ HomeController.cs  
├─ Models/  
│   └─ MyDataModel.cs  
├─ Views/  
│   ├─ Home/  
│   │   └─ Index.cshtml  
│   │   └─ About.cshtml  
│   └─ Shared/  
│       └─ _Layout.cshtml  
├─ Program.cs  
├─ appsettings.json  
└─ ...
```

Ví dụ MVC đơn giản:

`Controllers/HomeController.cs`:

```
using Microsoft.AspNetCore.Mvc;  
using MyMvcApp.Models;  
  
namespace MyMvcApp.Controllers  
{  
    public class HomeController : Controller  
    {  
        public IActionResult Index()  
        {  
            // Truyền dữ liệu đến View  
            ViewData["Message"] = "Chào mừng bạn đến với ứng dụng  
MVC!";  
            return View();  
        }  
  
        public IActionResult About()  
        {
```

```

        var model = new MyDataModel { Name = "ASP.NET Core",
Version = "8.0" };
        return View(model);
    }
}
}

```

Models/MyDataModel.cs:

```

namespace MyMvcApp.Models
{
    public class MyDataModel
    {
        public string Name { get; set; }
        public string Version { get; set; }
    }
}

```

Views/Home/Index.cshtml:

```

@{
    ViewData["Title"] = "Trang chủ";
}

<div class="text-center">
    <h1 class="display-4">@ViewData["Message"]</h1>
    <p>Tìm hiểu thêm về <a
href="https://docs.microsoft.com/aspnet/core/mvc">xây dựng ứng dụng
MVC với ASP.NET Core</a>.</p>
</div>

```

Views/Home/About.cshtml:

```
@model MyMvcApp.Models.MyDataModel
@{
    ViewData["Title"] = "Giới thiệu";
}

<h2>Giới thiệu về @Model.Name</h2>
<p>Phiên bản: @Model.Version</p>
```

Trong `Program.cs`, bạn cần thêm `app.MapControllerRoute()` để kích hoạt MVC:

```
// ...
app.UseRouting();
app.UseAuthorization();

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");

app.Run();
```

Khi nào sử dụng Razor Pages, khi nào sử dụng MVC?

- **Razor Pages:** Thích hợp cho các ứng dụng dựa trên trang, nơi mỗi trang có logic riêng biệt và không có sự tương tác phức tạp giữa các trang. Phù hợp cho các ứng dụng CRUD đơn giản, các trang quản trị, hoặc các trang landing page.
- **MVC:** Thích hợp cho các ứng dụng lớn hơn, phức tạp hơn, nơi cần sự tách biệt rõ ràng giữa các thành phần và có nhiều logic nghiệp vụ phức tạp. Phù hợp cho các ứng dụng thương mại điện tử, hệ thống quản lý phức tạp, hoặc các ứng dụng có nhiều API.

Bạn có thể sử dụng cả hai mô hình trong cùng một ứng dụng ASP.NET Core, tùy thuộc vào yêu cầu cụ thể của từng phần.

1.7 Làm việc với Static Files

Trong ứng dụng web, **Static Files** (tệp tĩnh) là những tệp được phục vụ trực tiếp cho trình duyệt mà không cần xử lý phía máy chủ. Chúng bao gồm các tệp HTML, CSS, JavaScript, hình ảnh, font chữ, v.v. ASP.NET Core cung cấp một middleware mạnh mẽ để phục vụ các tệp tĩnh này.

Thư mục `wwwroot`:

Theo quy ước, tất cả các tệp tĩnh trong ASP.NET Core đều được đặt trong thư mục `wwwroot` ở thư mục gốc của dự án. Khi ứng dụng chạy, thư mục `wwwroot` được coi là thư mục gốc web (web root) và các tệp bên trong nó có thể được truy cập trực tiếp thông qua URL.

Ví dụ, nếu bạn có một tệp `site.css` trong `wwwroot/css/`, bạn có thể truy cập nó từ trình duyệt bằng URL `/css/site.css`.

Kích hoạt Static Files Middleware:

Để ASP.NET Core có thể phục vụ các tệp tĩnh, bạn cần kích hoạt `Static Files Middleware` trong `Program.cs` (hoặc `Startup.cs`):

```
var builder = WebApplication.CreateBuilder(args);

// ... (Add services)

var app = builder.Build();

// ... (Configure middleware)

// Kích hoạt Static Files Middleware
app.UseStaticFiles();

// ... (Các middleware khác như UseRouting, UseAuthorization,
MapControllerRoute)

app.Run();
```

Phương thức mở rộng `UseStaticFiles()` sẽ thêm `StaticFilesMiddleware` vào pipeline xử lý yêu cầu. Middleware này sẽ kiểm tra xem yêu cầu đến có khớp với một tệp tĩnh trong `wwwroot` hay không. Nếu có, nó sẽ phục vụ tệp đó và kết thúc xử lý yêu cầu; nếu không, nó sẽ chuyển yêu cầu cho middleware tiếp theo.

Ví dụ về cấu trúc thư mục `wwwroot`:

```
wwwroot/  
├─ css/  
│   └─ site.css  
├─ js/  
│   └─ site.js  
├─ images/  
│   └─ logo.png  
└─ lib/  
    ├─ bootstrap/  
    └─ jquery/
```

Cách tham chiếu Static Files trong Razor Views:

Bạn có thể tham chiếu các tệp tĩnh trong các tệp `.cshtml` (Razor Views) bằng cách sử dụng đường dẫn tương đối từ thư mục gốc web (`wwwroot`).

```
<!DOCTYPE html>  
<html>  
<head>  
    <meta charset="utf-8" />  
    <title>Ứng dụng Static Files</title>  
    <link rel="stylesheet" href="~/css/site.css" />  
</head>  
<body>  
    <h1>Chào mừng đến với Static Files!</h1>  
      
    <script src="~/js/site.js"></script>  
</body>  
</html>
```

Lưu ý ký tự `~` ở đầu đường dẫn. Trong ASP.NET Core, `~` được dịch thành đường dẫn gốc của ứng dụng, giúp bạn tham chiếu các tệp tĩnh một cách linh hoạt, bất kể ứng dụng được triển khai ở đâu.

Phục vụ tệp tĩnh từ các thư mục khác (không phải `wwwroot`):

Trong một số trường hợp, bạn có thể muốn phục vụ các tệp tĩnh từ một thư mục khác ngoài `wwwroot`. Bạn có thể làm điều này bằng cách cấu hình `StaticFilesMiddleware`:

```
var builder = WebApplication.CreateBuilder(args);

// ...

var app = builder.Build();

// ...

// Phục vụ tệp tĩnh từ thư mục 'MyFiles' dưới URL '/my-static-files'
app.UseStaticFiles(new StaticFileOptions
{
    FileProvider = new PhysicalFileProvider(
        Path.Combine(builder.Environment.ContentRootPath,
            "MyFiles")),
    RequestPath = "/my-static-files"
});

// ...

app.Run();
```

Với cấu hình trên, nếu bạn có một tệp `document.pdf` trong thư mục `MyFiles`, bạn có thể truy cập nó bằng URL `/my-static-files/document.pdf`.

Việc quản lý và phục vụ tệp tĩnh là một phần cơ bản của bất kỳ ứng dụng web nào, và ASP.NET Core cung cấp một cách tiếp cận đơn giản và hiệu quả để thực hiện điều này.

1.8 Xử lý lỗi cơ bản

Trong quá trình phát triển và triển khai ứng dụng web, việc xử lý lỗi là một khía cạnh quan trọng để đảm bảo ứng dụng hoạt động ổn định và cung cấp trải nghiệm tốt cho người dùng. ASP.NET Core cung cấp nhiều cơ chế để xử lý lỗi, từ các lỗi phát sinh trong quá trình phát triển đến các lỗi trong môi trường sản xuất.

1.8.1 Môi trường phát triển và sản xuất

ASP.NET Core sử dụng khái niệm môi trường (environment) để điều chỉnh hành vi của ứng dụng. Hai môi trường phổ biến nhất là **Development** (phát triển) và **Production** (sản xuất).

- **Development** : Trong môi trường này, ứng dụng thường hiển thị thông tin lỗi chi tiết (ví dụ: stack trace) để giúp nhà phát triển dễ dàng gỡ lỗi. Điều này không an toàn trong môi trường sản xuất.
- **Production** : Trong môi trường này, ứng dụng nên hiển thị các trang lỗi thân thiện với người dùng và không tiết lộ thông tin nhạy cảm về lỗi.

Bạn có thể kiểm tra môi trường hiện tại bằng `app.Environment.IsDevelopment()` hoặc `app.Environment.IsProduction()`.

1.8.2 Middleware xử lý lỗi

ASP.NET Core cung cấp các middleware chuyên dụng để xử lý lỗi trong pipeline xử lý yêu cầu.

- **UseDeveloperExceptionPage()** : Middleware này chỉ nên được sử dụng trong môi trường phát triển. Khi một ngoại lệ xảy ra, nó sẽ hiển thị một trang lỗi chi tiết với stack trace, thông tin request, và các biến môi trường, rất hữu ích cho việc gỡ lỗi.

```
// Trong Program.cs
if (app.Environment.IsDevelopment())
{
    app.UseDeveloperExceptionPage();
}
```

- **UseExceptionHandler()**: Middleware này được sử dụng trong môi trường sản xuất. Khi một ngoại lệ xảy ra, nó sẽ chuyển hướng yêu cầu đến một đường dẫn (path) được chỉ định, nơi bạn có thể hiển thị một trang lỗi tùy chỉnh và thân thiện với người dùng.

```
// Trong Program.cs
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error"); // Chuyển hướng
    // The default HSTS value is 30 days. You may want to
    // change this for production scenarios, see
    // https://aka.ms/aspnetcore-hsts.
    app.UseHsts();
}
```

Bạn cần tạo một Controller và Action để xử lý đường dẫn `/Home/Error`:

Controllers/HomeController.cs:

```
using Microsoft.AspNetCore.Mvc;
using System.Diagnostics;
using MyWebApp.Models;

public class HomeController : Controller
{
    [ResponseCache(Duration = 0, Location =
    ResponseCacheLocation.None, NoStore = true)]
    public IActionResult Error()
    {
        return View(new ErrorViewModel { RequestId =
        Activity.Current?.Id ?? HttpContext.TraceIdentifier });
    }
}
```

Models/ErrorViewModel.cs:

```

using System;

namespace MyWebApp.Models
{
    public class ErrorViewModel
    {
        public string RequestId { get; set; }

        public bool ShowRequestId =>
            !string.IsNullOrEmpty(RequestId);
    }
}

```

Views/Shared/Error.cshtml:

```

@model ErrorViewModel
@{
    ViewData["Title"] = "Lỗi";
}

<h1 class="text-danger">Lỗi.</h1>
<h2 class="text-danger">Đã xảy ra lỗi khi xử lý yêu cầu của
bạn.</h2>

@if (Model.ShowRequestId)
{
    <p>
        <strong>Request ID:</strong>
        <code>@Model.RequestId</code>
    </p>
}

<h3>Chế độ phát triển</h3>
<p>
    Chuyển sang môi trường <strong>Development</strong> sẽ
    hiển thị thông tin chi tiết hơn về lỗi đã xảy ra.
</p>
<p>

```

Môi trường **Development** không nên được bật cho các ứng dụng đã triển khai. Nó có thể dẫn đến việc hiển thị thông tin nhạy cảm từ các ngoại lệ cho người dùng cuối. Để gỡ lỗi, hãy bật môi trường **Development** bằng cách đặt biến môi trường `ASPNETCORE_ENVIRONMENT` thành `Development` và khởi động lại ứng dụng.

1.8.3 Xử lý lỗi HTTP Status Code

ASP.NET Core cũng có thể xử lý các lỗi HTTP status code (ví dụ: 404 Not Found, 401 Unauthorized) bằng middleware `UseStatusCodePages()`.

```
// Trong Program.cs, sau UseRouting()
app.UseStatusCodePages(); // Hiển thị trang mặc định cho các mã
trạng thái HTTP

// Hoặc tùy chỉnh với lambda
app.UseStatusCodePages(async context =>
{
    context.Response.ContentType = "text/plain";
    await context.Response.WriteAsync(
        "Status code page, status code: " +
        context.Response.StatusCode);
});

// Hoặc chuyển hướng đến một trang lỗi cụ thể
app.UseStatusCodePagesWithRedirects("/Home/Error?statusCode={0}");
// Sau đó, trong Error action của HomeController, bạn có thể lấy
statusCode từ query string
```

1.8.4 try-catch blocks

Đối với các khối code cụ thể mà bạn biết có thể phát sinh ngoại lệ, bạn nên sử dụng khối `try-catch` để xử lý chúng một cách cục bộ.

```
public IActionResult DoSomethingDangerous()
```

```

{
    try
    {
        // Code có thể gây ra lỗi
        int zero = 0;
        int result = 10 / zero; // Sẽ gây ra DivideByZeroException
        return Ok("Thành công!");
    }
    catch (DivideByZeroException ex)
    {
        // Xử lý lỗi chia cho 0
        // Ghi log lỗi (sẽ học ở phần Logging)
        Console.WriteLine($"Lỗi chia cho 0: {ex.Message}");
        return BadRequest("Không thể chia cho 0.");
    }
    catch (Exception ex)
    {
        // Xử lý các lỗi khác
        Console.WriteLine($"Đã xảy ra lỗi: {ex.Message}");
        return StatusCode(500, "Đã xảy ra lỗi nội bộ máy chủ.");
    }
}

```

Việc kết hợp các middleware xử lý lỗi toàn cục với các khối `try-catch` cục bộ sẽ giúp bạn xây dựng một ứng dụng ASP.NET Core mạnh mẽ và có khả năng phục hồi tốt.

2. Middle Level: Phát triển ứng dụng nâng cao

Ở cấp độ này, chúng ta sẽ đi sâu vào các kỹ thuật phát triển ứng dụng ASP.NET Core nâng cao hơn, tập trung vào việc quản lý dữ liệu, xác thực, xây dựng API, ghi log và kiểm thử.

2.1 Làm việc với dữ liệu (Entity Framework Core)

Entity Framework Core (EF Core) là một ORM (Object-Relational Mapper) nhẹ, đa nền tảng và mã nguồn mở của Microsoft. Nó cho phép các nhà phát triển .NET làm việc với cơ sở dữ liệu bằng cách sử dụng các đối tượng .NET (được gọi là "entities") thay vì phải viết các câu lệnh SQL trực tiếp. EF Core giúp đơn giản hóa việc tương tác với cơ sở dữ liệu, giảm thiểu code lặp lại và tăng năng suất.

Các khái niệm chính trong EF Core:

1. **DbContext**: Là lớp chính trong EF Core, đại diện cho một phiên làm việc với cơ sở dữ liệu. Nó là cầu nối giữa các entities của bạn và cơ sở dữ liệu. DbContext chịu trách nhiệm truy vấn, theo dõi thay đổi, và lưu các thay đổi vào cơ sở dữ liệu.
2. **Entity**: Là các lớp C# đơn giản đại diện cho các bảng trong cơ sở dữ liệu của bạn. Mỗi thuộc tính của entity thường ánh xạ tới một cột trong bảng.
3. **DbSet**: Một thuộc tính trong DbContext đại diện cho một tập hợp các entities của một kiểu cụ thể trong cơ sở dữ liệu. Nó cho phép bạn thực hiện các thao tác CRUD (Create, Read, Update, Delete) trên các entities đó.
4. **Migrations**: Là một tính năng của EF Core cho phép bạn quản lý lược đồ cơ sở dữ liệu (database schema) theo thời gian. Khi bạn thay đổi mô hình dữ liệu (thêm/sửa/xóa thuộc tính trong entity), bạn có thể tạo một migration để cập nhật lược đồ cơ sở dữ liệu cho phù hợp.

Các bước cơ bản để sử dụng EF Core:

Bước 1: Cài đặt các gói NuGet cần thiết

Bạn cần cài đặt các gói NuGet sau vào dự án của mình. Ví dụ, nếu bạn sử dụng SQL Server:

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

- `Microsoft.EntityFrameworkCore.SqlServer`: Cung cấp nhà cung cấp cơ sở dữ liệu cho SQL Server.
- `Microsoft.EntityFrameworkCore.Tools`: Cung cấp các công cụ dòng lệnh (CLI) để quản lý migrations.

Bước 2: Định nghĩa Entity Model

Tạo các lớp C# đại diện cho dữ liệu của bạn. Ví dụ, một lớp `Product`:

```
// Models/Product.cs
using System.ComponentModel.DataAnnotations;

namespace MyWebApp.Models
{
    public class Product
    {
        public int Id { get; set; } // Khóa chính (Primary Key)

        [Required] // Thuộc tính này là bắt buộc
        [StringLength(100)] // Chiều dài tối đa của chuỗi
        public string Name { get; set; }

        public decimal Price { get; set; }

        public int Stock { get; set; }
    }
}
```

Bước 3: Tạo DbContext

Tạo một lớp kế thừa từ `DbContext` và định nghĩa các `DbSet` cho các entities của bạn:

```
// Data/ApplicationDbContext.cs
using Microsoft.EntityFrameworkCore;
using MyWebApp.Models;

namespace MyWebApp.Data
{
    public class ApplicationDbContext : DbContext
    {
        public
        ApplicationDbContext(DbContextOptions<ApplicationDbContext>
options)
            : base(options)
        {
        }
    }
}
```

```

        public DbSet<Product> Products { get; set; } // Ánh xạ tới
        bảng Products trong DB

        // Bạn có thể cấu hình mô hình ở đây nếu cần
        protected override void OnModelCreating(ModelBuilder
        modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            // Ví dụ: Cấu hình khóa chính composite, chỉ mục, v.v.
            // modelBuilder.Entity<Product>().HasKey(p => p.Id);
        }
    }
}

```

Bước 4: Cấu hình DbContext trong Program.cs

Đăng ký ApplicationDbContext vào hệ thống Dependency Injection và cấu hình chuỗi kết nối cơ sở dữ liệu:

```

// Program.cs
using Microsoft.EntityFrameworkCore;
using MyWebApp.Data;

var builder = WebApplication.CreateBuilder(args);

// Đọc chuỗi kết nối từ appsettings.json
var connectionString =
builder.Configuration.GetConnectionString("DefaultConnection");

// Đăng ký DbContext với SQL Server
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(connectionString));

// ... (Các dịch vụ khác)

var app = builder.Build();

// ... (Middleware)

```

```
app.Run();
```

Trong tệp `appsettings.json`, bạn cần thêm chuỗi kết nối:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Server=
(localdb)\\mssqllocaldb;Database=MyWebAppDb;Trusted_Connection=True
;MultipleActiveResultSets=true"
  },
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning"
    }
  },
  "AllowedHosts": "*"
}
```

Bước 5: Tạo và áp dụng Migrations

Sau khi định nghĩa entities và `DbContext`, bạn cần tạo migration để EF Core tạo ra lược đồ cơ sở dữ liệu tương ứng. Mở Terminal (hoặc Command Prompt) trong thư mục gốc của dự án và chạy các lệnh sau:

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

- `dotnet ef migrations add InitialCreate`: Tạo một migration mới có tên `InitialCreate`. Lệnh này sẽ tạo ra một thư mục `Migrations` trong dự án của bạn, chứa các tệp code mô tả các thay đổi lược đồ cơ sở dữ liệu.
- `dotnet ef database update`: Áp dụng các migration đang chờ xử lý vào cơ sở dữ liệu. Lệnh này sẽ tạo cơ sở dữ liệu (nếu chưa tồn tại) và các bảng dựa trên các entities của bạn.

Bước 6: Sử dụng DbContext trong Controller/Razor Page

Bạn có thể inject `ApplicationDbContext` vào Controller hoặc Razor Page bằng Dependency Injection:

```
// Controllers/ProductsController.cs
using Microsoft.AspNetCore.Mvc;
using MyWebApp.Data;
using MyWebApp.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace MyWebApp.Controllers
{
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // Lấy tất cả sản phẩm
        public async Task<IActionResult> Index()
        {
            List<Product> products = await
            _context.Products.ToListAsync();
            return View(products);
        }

        // ... (Các thao tác CRUD khác sẽ được trình bày chi tiết ở
        phần sau)
    }
}
```

EF Core là một công cụ mạnh mẽ giúp bạn tương tác với cơ sở dữ liệu một cách hiệu quả và an toàn trong các ứng dụng ASP.NET Core. Việc nắm vững các khái niệm và cách sử dụng cơ bản của nó là rất quan trọng cho các nhà phát triển Middle Level.

2.2 CRUD Operations

CRUD là viết tắt của **Create, Read, Update, Delete** – bốn thao tác cơ bản mà hầu hết các ứng dụng tương tác với dữ liệu đều phải thực hiện. Trong phần này, chúng ta sẽ tìm hiểu cách thực hiện các thao tác CRUD sử dụng Entity Framework Core trong ứng dụng ASP.NET Core MVC.

Chúng ta sẽ tiếp tục với ví dụ `Product` từ phần trước.

Bước 1: Tạo Controller cho Product

Nếu bạn chưa có, hãy tạo một `ProductsController`:

```
// Controllers/ProductsController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyWebApp.Data;
using MyWebApp.Models;
using System.Linq;
using System.Threading.Tasks;

namespace MyWebApp.Controllers
{
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;

        public ProductsController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: Products
        public async Task<IActionResult> Index()
        {
            return View(await _context.Products.ToListAsync());
        }

        // GET: Products/Details/5
        public async Task<IActionResult> Details(int? id)
        {

```

```

        if (id == null)
        {
            return NotFound();
        }

        var product = await _context.Products
            .FirstOrDefaultAsync(m => m.Id == id);
        if (product == null)
        {
            return NotFound();
        }

        return View(product);
    }

    // GET: Products/Create
    public IActionResult Create()
    {
        return View();
    }

    // POST: Products/Create
    [HttpPost]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult>
Create([Bind("Id,Name,Price,Stock")] Product product)
    {
        if (ModelState.IsValid)
        {
            _context.Add(product);
            await _context.SaveChangesAsync();
            return RedirectToAction(nameof(Index));
        }
        return View(product);
    }

    // GET: Products/Edit/5
    public async Task<IActionResult> Edit(int? id)
    {
        if (id == null)

```

```

    {
        return NotFound();
    }

    var product = await _context.Products.FindAsync(id);
    if (product == null)
    {
        return NotFound();
    }
    return View(product);
}

// POST: Products/Edit/5
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(int id,
[Bind("Id,Name,Price,Stock")] Product product)
{
    if (id != product.Id)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {
        try
        {
            _context.Update(product);
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!ProductExists(product.Id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
    }
}

```



```

        }
        return RedirectToAction(nameof(Index));
    }
    return View(product);
}

// GET: Products/Delete/5
public async Task<IActionResult> Delete(int? id)
{
    if (id == null)
    {
        return NotFound();
    }

    var product = await _context.Products
        .FirstOrDefaultAsync(m => m.Id == id);
    if (product == null)
    {
        return NotFound();
    }

    return View(product);
}

// POST: Products/Delete/5
[HttpPost, ActionName("Delete")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> DeleteConfirmed(int id)
{
    var product = await _context.Products.FindAsync(id);
    if (product != null)
    {
        _context.Products.Remove(product);
    }

    await _context.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}

private bool ProductExists(int id)

```

```

        {
            return _context.Products.Any(e => e.Id == id);
        }
    }
}

```

Bước 2: Tạo Views tương ứng

Bạn cần tạo các Views (.cshtml files) trong thư mục `Views/Products/` tương ứng với mỗi Action trong `ProductsController`.

`Views/Products/Index.cshtml` (Hiển thị danh sách sản phẩm):

```

@model IEnumerable<MyWebApp.Models.Product>

@{
    ViewData["Title"] = "Sản phẩm";
}

<h1>Danh sách sản phẩm</h1>

<p>
    <a asp-action="Create">Tạo mới</a>
</p>
<table class="table">
    <thead>
        <tr>
            <th>
                @Html.DisplayNameFor(model => model.Name)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Price)
            </th>
            <th>
                @Html.DisplayNameFor(model => model.Stock)
            </th>
        </tr>
    </thead>

```

```

        <tbody>
@foreach (var item in Model) {
    <tr>
        <td>
            @Html.DisplayFor(modelItem => item.Name)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.Price)
        </td>
        <td>
            @Html.DisplayFor(modelItem => item.Stock)
        </td>
        <td>
            <a asp-action="Edit" asp-route-
id="@item.Id">Sửa</a> |
            <a asp-action="Details" asp-route-id="@item.Id">Chi
tiết</a> |
            <a asp-action="Delete" asp-route-
id="@item.Id">Xóa</a>
        </td>
    </tr>
}
        </tbody>
    </table>

```

Views/Products/Details.cshtml (Hiển thị chi tiết sản phẩm):

```

@model MyWebApp.Models.Product

@{
    ViewData["Title"] = "Chi tiết sản phẩm";
}

<h1>Chi tiết sản phẩm</h1>

<div>
    <h4>Product</h4>
    <hr />
    <dl class="row">

```

```

        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Price)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Price)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Stock)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Stock)
        </dd>
    </dl>
</div>
<div>
    <a asp-action="Edit" asp-route-id="@Model.Id">Sửa</a> |
    <a asp-action="Index">Quay lại danh sách</a>
</div>

```

Views/Products/Create.cshtml (Form tạo sản phẩm mới):

```

@model MyWebApp.Models.Product

@{
    ViewData["Title"] = "Tạo sản phẩm mới";
}

<h1>Tạo sản phẩm mới</h1>

<h4>Product</h4>
<hr />
<div class="row">
    <div class="col-md-4">

```

```

        <form asp-action="Create">
            <div asp-validation-summary="ModelOnly" class="text-
danger"></div>
            <div class="form-group">
                <label asp-for="Name" class="control-label">
</label>
                <input asp-for="Name" class="form-control" />
                <span asp-validation-for="Name" class="text-
danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Price" class="control-label">
</label>
                <input asp-for="Price" class="form-control" />
                <span asp-validation-for="Price" class="text-
danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Stock" class="control-label">
</label>
                <input asp-for="Stock" class="form-control" />
                <span asp-validation-for="Stock" class="text-
danger"></span>
            </div>
            <div class="form-group">
                <input type="submit" value="Tạo" class="btn btn-
primary" />
            </div>
        </form>
    </div>

<div>
    <a asp-action="Index">Quay lại danh sách</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

Views/Products/Edit.cshtml (Form chỉnh sửa sản phẩm):

```
@model MyWebApp.Models.Product

@{
    ViewData["Title"] = "Chỉnh sửa sản phẩm";
}

<h1>Chỉnh sửa sản phẩm</h1>

<h4>Product</h4>
<hr />
<div class="row">
    <div class="col-md-4">
        <form asp-action="Edit">
            <div asp-validation-summary="ModelOnly" class="text-
danger"></div>
            <input type="hidden" asp-for="Id" />
            <div class="form-group">
                <label asp-for="Name" class="control-label">
</label>

                <input asp-for="Name" class="form-control" />
                <span asp-validation-for="Name" class="text-
danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Price" class="control-label">
</label>

                <input asp-for="Price" class="form-control" />
                <span asp-validation-for="Price" class="text-
danger"></span>
            </div>
            <div class="form-group">
                <label asp-for="Stock" class="control-label">
</label>

                <input asp-for="Stock" class="form-control" />
                <span asp-validation-for="Stock" class="text-
danger"></span>
            </div>
            <div class="form-group">
```

```

        <input type="submit" value="Lưu" class="btn btn-
primary" />
    </div>
</form>
</div>
</div>

<div>
    <a asp-action="Index">Quay lại danh sách</a>
</div>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

Views/Products/Delete.cshtml (Xác nhận xóa sản phẩm):

```

@model MyWebApp.Models.Product

@{
    ViewData["Title"] = "Xóa sản phẩm";
}

<h1>Xóa sản phẩm</h1>

<h3>Bạn có chắc chắn muốn xóa sản phẩm này không?</h3>
<div>
    <h4>Product</h4>
    <hr />
    <dl class="row">
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Name)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Name)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Price)
        </dt>
    </dl>

```

```

        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Price)
        </dd>
        <dt class="col-sm-2">
            @Html.DisplayNameFor(model => model.Stock)
        </dt>
        <dd class="col-sm-10">
            @Html.DisplayFor(model => model.Stock)
        </dd>
    </dl>

    <form asp-action="Delete">
        <input type="hidden" asp-for="Id" />
        <input type="submit" value="Xóa" class="btn btn-danger" />
    </form>
    <a asp-action="Index">Quay lại danh sách</a>
</div>

```

Các ví dụ trên minh họa cách thực hiện các thao tác CRUD cơ bản trong ASP.NET Core MVC sử dụng Entity Framework Core. Bạn có thể áp dụng các nguyên tắc này để quản lý dữ liệu cho bất kỳ entity nào trong ứng dụng của mình.

2.3 Validation và Model Binding

Trong các ứng dụng web, việc đảm bảo dữ liệu người dùng nhập vào là hợp lệ và được ánh xạ đúng cách vào các đối tượng C# là rất quan trọng. ASP.NET Core cung cấp các tính năng mạnh mẽ cho **Model Binding** và **Validation** để đơn giản hóa quá trình này.

2.3.1 Model Binding

Model Binding là quá trình ASP.NET Core ánh xạ dữ liệu từ các yêu cầu HTTP (ví dụ: form fields, query string parameters, route data, JSON/XML body) vào các tham số của action method hoặc các thuộc tính của một model. Điều này giúp bạn làm việc với dữ liệu người dùng một cách thuận tiện bằng các đối tượng C# thay vì phải truy cập trực tiếp vào `HttpRequest`.

Cách hoạt động:

Khi một yêu cầu HTTP đến, Model Binder sẽ:

1. Tìm kiếm dữ liệu trong các nguồn khác nhau (form, route, query string, header, body).
2. Chuyển đổi dữ liệu từ định dạng chuỗi sang kiểu dữ liệu .NET phù hợp (ví dụ: "123" thành `int`, "true" thành `bool`).
3. Ánh xạ dữ liệu đã chuyển đổi vào các thuộc tính của model hoặc tham số của action method.

Ví dụ:

Giả sử bạn có một form HTML với các trường `Name`, `Price`, `Stock` và một action method trong Controller:

```
<!-- Trong View (ví dụ: Products/Create.cshtml) -->
<form asp-action="Create" method="post">
    <input type="text" name="Name" />
    <input type="number" name="Price" />
    <input type="number" name="Stock" />
    <button type="submit">Tạo</button>
</form>
```

```
// Trong ProductsController.cs
[HttpPost]
public async Task<IActionResult> Create(Product product)
{
    // Model Binder sẽ tự động ánh xạ các giá trị từ form vào đối tượng 'product'
    // product.Name sẽ chứa giá trị từ input 'Name'
    // product.Price sẽ chứa giá trị từ input 'Price'
    // product.Stock sẽ chứa giá trị từ input 'Stock'
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
}
```

```
    return View(product);  
}
```

Trong ví dụ trên, khi form được gửi đi, Model Binder sẽ tự động tạo một đối tượng `Product` và điền các thuộc tính `Name`, `Price`, `Stock` từ dữ liệu form. Bạn không cần phải tự mình phân tích cú pháp request.

2.3.2 Validation (Xác thực dữ liệu)

Validation là quá trình kiểm tra xem dữ liệu người dùng nhập vào có tuân thủ các quy tắc nghiệp vụ và định dạng mong muốn hay không. ASP.NET Core cung cấp một hệ thống validation mạnh mẽ dựa trên Data Annotations và tích hợp chặt chẽ với Model Binding.

Sử dụng Data Annotations:

Bạn có thể thêm các thuộc tính (attributes) Data Annotation vào các thuộc tính của model để định nghĩa các quy tắc xác thực. Các thuộc tính phổ biến bao gồm:

- `[Required]` : Thuộc tính là bắt buộc.
- `[StringLength(maxLength, MinimumLength = minLength)]` : Giới hạn độ dài của chuỗi.
- `[Range(minimum, maximum)]` : Giới hạn giá trị số.
- `[EmailAddress]` : Kiểm tra định dạng email.
- `[Url]` : Kiểm tra định dạng URL.
- `[CreditCard]` : Kiểm tra định dạng số thẻ tín dụng.
- `[Compare("OtherProperty")]` : So sánh giá trị với một thuộc tính khác (ví dụ: xác nhận mật khẩu).
- `[RegularExpression("pattern")]` : Kiểm tra bằng biểu thức chính quy.

Ví dụ với `Product` Model:

```
// Models/Product.cs  
using System.ComponentModel.DataAnnotations;  
  
namespace MyWebApp.Models  
{
```

```

public class Product
{
    public int Id { get; set; }

    [Required(ErrorMessage =
        "Tên sản phẩm là bắt buộc.")]
    [StringLength(100, MinimumLength = 3, ErrorMessage = "Tên
sản phẩm phải có từ 3 đến 100 ký tự.")]
    public string Name { get; set; }

    [Required(ErrorMessage = "Giá sản phẩm là bắt buộc.")]
    [Range(0.01, 10000.00, ErrorMessage = "Giá sản phẩm phải
nằm trong khoảng từ 0.01 đến 10000.00.")]
    [DataType(DataType.Currency)]
    public decimal Price { get; set; }

    [Required(ErrorMessage = "Số lượng tồn kho là bắt buộc.")]
    [Range(0, 100000, ErrorMessage = "Số lượng tồn kho phải nằm
trong khoảng từ 0 đến 100000.")]
    public int Stock { get; set; }
}

```

Kiểm tra Validation trong Controller:

Sau khi Model Binding diễn ra, ASP.NET Core sẽ tự động chạy các quy tắc validation được định nghĩa bằng Data Annotations. Kết quả của quá trình validation được lưu trữ trong đối tượng ModelState.

Bạn có thể kiểm tra `ModelState.IsValid` trong action method để xác định xem dữ liệu có hợp lệ hay không:

```

// Trong ProductsController.cs
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult>
Create([Bind("Id,Name,Price,Stock")] Product product)

```

```

{
    if (ModelState.IsValid) // Kiểm tra xem dữ liệu model có hợp lệ
    không
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    // Nếu không hợp lệ, quay lại View với các thông báo lỗi
    return View(product);
}

```

Hiển thị thông báo lỗi trong View:

Trong Razor Views, bạn có thể sử dụng các Tag Helpers để hiển thị thông báo lỗi validation cho người dùng.

- **asp-validation-summary**: Hiển thị tất cả các thông báo lỗi validation cho toàn bộ model.
- **asp-validation-for**: Hiển thị thông báo lỗi validation cho một thuộc tính cụ thể.

```

<!-- Trong Views/Products/Create.cshtml hoặc Edit.cshtml -->
<form asp-action="Create">
    <div asp-validation-summary="ModelOnly" class="text-danger">
</div>
    <div class="form-group">
        <label asp-for="Name" class="control-label"></label>
        <input asp-for="Name" class="form-control" />
        <span asp-validation-for="Name" class="text-danger"></span>
    </div>
    <div class="form-group">
        <label asp-for="Price" class="control-label"></label>
        <input asp-for="Price" class="form-control" />
        <span asp-validation-for="Price" class="text-danger">
</span>
    </div>
    <div class="form-group">
        <label asp-for="Stock" class="control-label"></label>

```

```

        <input asp-for="Stock" class="form-control" />
        <span asp-validation-for="Stock" class="text-danger">
</span>
    </div>
    <div class="form-group">
        <input type="submit" value="Tạo" class="btn btn-primary" />
    </div>
</form>

@section Scripts {
    @{await Html.RenderPartialAsync("_ValidationScriptsPartial");}
}

```

`_ValidationScriptsPartial.cshtml` thường chứa các script JavaScript để thực hiện client-side validation, giúp cung cấp phản hồi ngay lập tức cho người dùng mà không cần gửi yêu cầu lên server.

Model Binding và Validation là những tính năng cốt lõi giúp bạn xây dựng các ứng dụng web an toàn và thân thiện với người dùng, đảm bảo rằng dữ liệu được xử lý đúng cách và hợp lệ trước khi được lưu trữ hoặc sử dụng.

2.4 Authentication và Authorization

Authentication (Xác thực) và **Authorization (Ủy quyền)** là hai khái niệm cốt lõi trong bảo mật ứng dụng web. Mặc dù thường đi đôi với nhau, chúng có những vai trò khác nhau:

- **Authentication:** Là quá trình xác minh danh tính của người dùng. Nó trả lời câu hỏi "Bạn là ai?". Ví dụ: đăng nhập bằng tên người dùng/mật khẩu, Google, Facebook.
- **Authorization:** Là quá trình xác định quyền truy cập của người dùng đã được xác thực vào một tài nguyên hoặc thực hiện một hành động cụ thể. Nó trả lời câu hỏi "Bạn được phép làm gì?". Ví dụ: người dùng A có quyền xem trang quản trị, người dùng B chỉ có quyền xem danh sách sản phẩm.

ASP.NET Core cung cấp một hệ thống mạnh mẽ và linh hoạt để quản lý cả Authentication và Authorization.

2.4.1 Authentication

ASP.NET Core hỗ trợ nhiều lược đồ xác thực (authentication schemes) khác nhau. Phổ biến nhất là **Cookie Authentication** cho các ứng dụng web truyền thống và **JWT Bearer Authentication** cho các API.

Cấu hình Authentication:

Để sử dụng Authentication, bạn cần cấu hình các dịch vụ xác thực trong `Program.cs`:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ... (Add services)

// Thêm dịch vụ xác thực
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login"; // Đường dẫn đến trang đăng nhập
        options.AccessDeniedPath = "/Account/AccessDenied"; // Đường dẫn khi truy cập bị từ chối
        options.ExpireTimeSpan = TimeSpan.FromMinutes(30); // Thời gian hết hạn của cookie
    });

builder.Services.AddControllersWithViews();

var app = builder.Build();

// ... (Configure middleware)

// Sử dụng Authentication middleware
app.UseAuthentication(); // Phải đứng trước UseAuthorization()
app.UseAuthorization();

// ... (Map routes)
```

```
app.Run();
```

- `AddAuthentication()`: Đăng ký dịch vụ xác thực và chỉ định lược đồ mặc định.
- `AddCookie()`: Cấu hình Cookie Authentication, đây là lược đồ phổ biến cho các ứng dụng web dựa trên session.

Ví dụ về đăng nhập (Login) và đăng xuất (Logout):

Giả sử bạn có một `AccountController` để xử lý các thao tác liên quan đến tài khoản.

`Controllers/AccountController.cs`:

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.AspNetCore.Authentication.Cookies;
using Microsoft.AspNetCore.Mvc;
using System.Collections.Generic;
using System.Security.Claims;
using System.Threading.Tasks;

namespace MyWebApp.Controllers
{
    public class AccountController : Controller
    {
        [HttpGet]
        public IActionResult Login()
        {
            return View();
        }

        [HttpPost]
        public async Task<IActionResult> Login(string username,
string password, string returnUrl)
        {
            // Đây là ví dụ đơn giản, trong thực tế bạn sẽ kiểm tra
            // thông tin đăng nhập từ database
            if (username == "admin" && password == "password")
            {
                var claims = new List<Claim>
                {
```

```

        new Claim(ClaimTypes.Name, username),
        new Claim(ClaimTypes.Role, "Administrator"), //
Gán vai trò
        new Claim("CustomClaim", "SomeValue") // Thêm
claim tùy chỉnh
    };

    var claimsIdentity = new ClaimsIdentity(
        claims,
CookieAuthenticationDefaults.AuthenticationScheme);

    var authProperties = new AuthenticationProperties
    {
        IsPersistent = true, // Ghi nhớ đăng nhập
        ExpiresUtc =
System.DateTimeOffset.UtcNow.AddMinutes(30)
    };

    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        new ClaimsPrincipal(claimsIdentity),
        authProperties);

    if (!string.IsNullOrEmpty(returnUrl) &&
Url.IsLocalUrl(returnUrl))
    {
        return Redirect(returnUrl);
    }
    else
    {
        return RedirectToAction("Index", "Home");
    }
}

ModelState.AddModelError(string.Empty, "Tên đăng nhập
hoặc mật khẩu không đúng.");
return View();
}

```



```

        [HttpPost]
        public async Task<IActionResult> Logout()
        {
            await
HttpContext.SignOutAsync(CookieAuthenticationDefaults.AuthenticationScheme);

            return RedirectToAction("Index", "Home");
        }

        [HttpGet]
        public IActionResult AccessDenied()
        {
            return View();
        }
    }
}

```

Views/Account/Login.cshtml:

```

@{
    ViewData["Title"] = "Đăng nhập";
}

<h1>Đăng nhập</h1>

<div class="row">
    <div class="col-md-4">
        <form asp-action="Login">
            <div asp-validation-summary="All" class="text-danger">
</div>
            <div class="form-group">
                <label for="username">Tên đăng nhập</label>
                <input type="text" name="username" class="form-
control" />
            </div>
            <div class="form-group">
                <label for="password">Mật khẩu</label>
                <input type="password" name="password" class="form-
control" />

```

```

        </div>
        <div class="form-group">
            <input type="submit" value="Đăng nhập" class="btn
btn-primary" />
        </div>
    </form>
</div>
</div>

```

2.4.2 Authorization

Sau khi người dùng được xác thực, Authorization sẽ kiểm tra xem người dùng đó có quyền truy cập vào tài nguyên được yêu cầu hay không. ASP.NET Core cung cấp các cách tiếp cận Authorization dựa trên vai trò (Role-based) và dựa trên chính sách (Policy-based).

Authorization dựa trên vai trò (Role-based Authorization):

Cách đơn giản nhất để ủy quyền là sử dụng thuộc tính `[Authorize]` và chỉ định các vai trò được phép truy cập.

```

// Controllers/AdminController.cs
using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;

namespace MyWebApp.Controllers
{
    [Authorize(Roles = "Administrator")] // Chỉ người dùng có vai
    trò "Administrator" mới có thể truy cập Controller này
    public class AdminController : Controller
    {
        public IActionResult Index()
        {
            return View();
        }

        [Authorize(Roles = "Editor")] // Chỉ người dùng có vai trò
        "Editor" mới có thể truy cập Action này
        public IActionResult EditContent()
        {

```

```
        return View();
    }
}
}
```

Nếu người dùng không được xác thực hoặc không có vai trò yêu cầu, họ sẽ bị chuyển hướng đến `AccessDeniedPath` đã cấu hình trong `AddCookie()`.

Authorization dựa trên chính sách (Policy-based Authorization):

Policy-based Authorization cung cấp một cách linh hoạt hơn để định nghĩa các quy tắc ủy quyền. Bạn có thể tạo các chính sách phức tạp dựa trên nhiều yêu cầu (requirements) khác nhau (ví dụ: vai trò, claim, hoặc các logic tùy chỉnh).

Bước 1: Định nghĩa Policy trong `Program.cs`:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ... (Add Authentication services)

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("RequireAdminRole", policy =>
        policy.RequireRole("Administrator"));

    options.AddPolicy("RequireAgeOver18", policy =>
        policy.RequireClaim(ClaimTypes.DateOfBirth, claim =>
        {
            // Logic kiểm tra tuổi từ claim DateOfBirth
            if (DateTime.TryParse(claim.Value, out DateTime dob))
            {
                return (DateTime.Now.Year - dob.Year) >= 18;
            }
            return false;
        }
    ));
});

builder.Services.AddControllersWithViews();
```

```
var app = builder.Build();  
  
// ...
```

Bước 2: Áp dụng Policy:

```
// Controllers/SomeController.cs  
using Microsoft.AspNetCore.Authorization;  
using Microsoft.AspNetCore.Mvc;  
  
namespace MyWebApp.Controllers  
{  
    public class SomeController : Controller  
    {  
        [Authorize(Policy = "RequireAdminRole")] // Áp dụng chính  
        sách "RequireAdminRole"  
        public IActionResult AdminDashboard()  
        {  
            return View();  
        }  
  
        [Authorize(Policy = "RequireAgeOver18")] // Áp dụng chính  
        sách "RequireAgeOver18"  
        public IActionResult AdultContent()  
        {  
            return View();  
        }  
    }  
}
```

Authentication và Authorization là những thành phần không thể thiếu để xây dựng các ứng dụng web an toàn và đáng tin cậy. Việc hiểu và áp dụng đúng cách các cơ chế này sẽ giúp bảo vệ ứng dụng và dữ liệu của bạn khỏi các truy cập trái phép.

2.5 API Development (RESTful APIs)

RESTful APIs (Representational State Transfer Application Programming Interfaces) là một kiến trúc phổ biến để xây dựng các dịch vụ web. Nó cho phép các ứng dụng khác (ví dụ: ứng dụng di động, ứng dụng frontend JavaScript như React/Angular/Vue, hoặc các dịch vụ backend khác) tương tác với dữ liệu và chức năng của ứng dụng của bạn thông qua HTTP.

ASP.NET Core cung cấp một cách mạnh mẽ và hiệu quả để xây dựng các RESTful APIs bằng cách sử dụng các Controller API.

Các nguyên tắc chính của REST:

1. **Stateless (Không trạng thái):** Mỗi yêu cầu từ client đến server phải chứa tất cả thông tin cần thiết để server hiểu và xử lý yêu cầu. Server không lưu trữ bất kỳ trạng thái nào của client giữa các yêu cầu.
2. **Client-Server:** Tách biệt rõ ràng giữa giao diện người dùng (client) và logic xử lý dữ liệu (server).
3. **Cacheable (Có thể lưu trữ cache):** Các phản hồi từ server có thể được đánh dấu là có thể lưu trữ cache hoặc không, giúp cải thiện hiệu suất.
4. **Layered System (Hệ thống phân lớp):** Client không cần biết liệu nó có đang kết nối trực tiếp với server cuối cùng hay thông qua một trung gian (ví dụ: load balancer, proxy).
5. **Uniform Interface (Giao diện đồng nhất):** Đây là nguyên tắc quan trọng nhất, bao gồm:
 - **Resource-based:** Mọi thứ đều là một tài nguyên (resource) và được xác định bằng một URI (Uniform Resource Identifier).
 - **Manipulation of resources through representations:** Client tương tác với tài nguyên thông qua các biểu diễn (representations) của chúng (ví dụ: JSON, XML).
 - **Self-descriptive messages:** Mỗi thông báo phải chứa đủ thông tin để client hiểu cách xử lý nó.
 - **Hypermedia as the Engine of Application State (HATEOAS):** Server cung cấp các liên kết (links) trong phản hồi để client có thể khám phá các hành động tiếp theo.

Xây dựng API Controller trong ASP.NET Core:

Để tạo một API Controller, bạn tạo một lớp kế thừa từ `ControllerBase` và thêm thuộc tính `[ApiController]`.

Ví dụ: API cho quản lý sản phẩm

Chúng ta sẽ tạo một API Controller để thực hiện các thao tác CRUD trên tài nguyên `Product`.

Controllers/ProductsApiController.cs:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using MyWebApp.Data;
using MyWebApp.Models;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace MyWebApp.Controllers
{
    [Route("api/[controller]")] // Định nghĩa route cho API
    Controller
    [ApiController] // Đánh dấu đây là một API Controller
    public class ProductsApiController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public ProductsApiController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/ProductsApi
        [HttpGet]
        public async Task<ActionResult<IEnumerable<Product>>>
        GetProducts()
        {
            return await _context.Products.ToListAsync();
        }

        // GET: api/ProductsApi/5
        [HttpGet("{id}")]
        public async Task<ActionResult<Product>> GetProduct(int id)
```

```

    {
        var product = await _context.Products.FindAsync(id);

        if (product == null)
        {
            return NotFound(); // Trả về 404 Not Found
        }

        return product; // Trả về 200 OK với đối tượng Product
    }

    // PUT: api/ProductsApi/5
    // Để cập nhật một tài nguyên hiện có
    [HttpPut("{id}")]
    public async Task<IActionResult> PutProduct(int id, Product
product)
    {
        if (id != product.Id)
        {
            return BadRequest(); // Trả về 400 Bad Request
        }

        _context.Entry(product).State = EntityState.Modified;

        try
        {
            await _context.SaveChangesAsync();
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!ProductExists(id))
            {
                return NotFound();
            }
            else
            {
                throw;
            }
        }
    }

```

```

        return NoContent(); // Trả về 204 No Content (thành
công nhưng không có nội dung trả về)
    }

    // POST: api/ProductsApi
    // Để tạo một tài nguyên mới
    [HttpPost]
    public async Task<ActionResult<Product>>
PostProduct(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();

        // Trả về 201 Created với URI của tài nguyên mới tạo
        return CreatedAtAction("GetProduct", new { id =
product.Id }, product);
    }

    // DELETE: api/ProductsApi/5
    [HttpDelete("{id}")]
    public async Task<IActionResult> DeleteProduct(int id)
    {
        var product = await _context.Products.FindAsync(id);
        if (product == null)
        {
            return NotFound();
        }

        _context.Products.Remove(product);
        await _context.SaveChangesAsync();

        return NoContent();
    }

    private bool ProductExists(int id)
    {
        return _context.Products.Any(e => e.Id == id);
    }
}

```


Giải thích các thuộc tính và phương thức:

- **[Route("api/[controller]")]**: Định nghĩa template route cho controller này. [controller] sẽ được thay thế bằng tên của controller (ví dụ: ProductsApi). Do đó, URI sẽ là api/ProductsApi.
- **[ApiController]**: Thuộc tính này cung cấp các tính năng tiện lợi cho API Controller, bao gồm:
 - **Automatic HTTP 400 responses**: Tự động trả về HTTP 400 Bad Request khi Model State không hợp lệ.
 - **Binding source parameter inference**: Tự động suy luận nguồn dữ liệu cho các tham số (ví dụ: từ route, query string, hoặc body).
 - **Problem details for error statuses**: Trả về định dạng ProblemDetails cho các lỗi HTTP.
- **ControllerBase**: Là lớp cơ sở cho các API Controller. Nó không bao gồm hỗ trợ View như Controller.
- **[HttpGet]**, **[HttpPost]**, **[HttpPut]**, **[HttpDelete]**: Các thuộc tính này ánh xạ các phương thức HTTP tương ứng đến các action method. Bạn có thể chỉ định thêm template route trong các thuộc tính này (ví dụ: [HttpGet("{id})]).
- **ActionResult<T>**: Kiểu trả về phổ biến cho các action method trong API Controller. Nó cho phép bạn trả về cả kiểu dữ liệu cụ thể (ví dụ: Product) hoặc các kết quả HTTP (ví dụ: NotFound(), BadRequest()).
- **CreatedAtAction()**: Một helper method trả về HTTP 201 Created, thường được sử dụng sau khi tạo thành công một tài nguyên mới. Nó bao gồm một header Location trỏ đến URI của tài nguyên mới tạo.

Cấu hình trong Program.cs:

Để kích hoạt các API Controller, bạn cần thêm `app.MapControllers();` vào `Program.cs`:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ... (Add services)

builder.Services.AddControllers(); // Thêm dịch vụ cho API
Controllers
```

```
var app = builder.Build();

// ... (Configure middleware)

app.UseRouting();
app.UseAuthorization();

app.MapControllers(); // Ánh xạ các API Controller

app.Run();
```

Kiểm thử API:

Bạn có thể sử dụng các công cụ như Postman, Insomnia, hoặc Swagger/OpenAPI để kiểm thử các API của mình. ASP.NET Core có tích hợp sẵn hỗ trợ cho Swagger/OpenAPI thông qua gói `Swashbuckle.AspNetCore`, giúp tự động tạo tài liệu API và giao diện người dùng để kiểm thử.

Cài đặt Swashbuckle.AspNetCore:

```
dotnet add package Swashbuckle.AspNetCore
```

Cấu hình Swashbuckle trong `Program.cs`:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ...

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer(); // Cần thiết cho
Swagger
builder.Services.AddSwaggerGen(); // Đăng ký dịch vụ Swagger

var app = builder.Build();

// ...
```

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger(); // Kích hoạt Swagger middleware
    app.UseSwaggerUI(); // Kích hoạt Swagger UI
}

// ...

app.MapControllers();

app.Run();
```

Sau khi cấu hình, bạn có thể truy cập Swagger UI tại `https://localhost:port/swagger` (hoặc `http://localhost:port/swagger`) để xem tài liệu API và gửi các yêu cầu kiểm thử.

Phát triển RESTful APIs là một kỹ năng quan trọng cho các nhà phát triển Middle Level, cho phép bạn xây dựng các dịch vụ backend mạnh mẽ và có thể tương tác với nhiều loại client khác nhau.

2.6 Logging và Monitoring

Logging (Ghi log) và **Monitoring (Giám sát)** là hai khía cạnh quan trọng trong việc phát triển và vận hành các ứng dụng. Logging giúp bạn ghi lại các sự kiện, lỗi, và thông tin quan trọng trong quá trình ứng dụng chạy, trong khi Monitoring giúp bạn theo dõi hiệu suất và trạng thái sức khỏe của ứng dụng theo thời gian thực.

2.6.1 Logging trong ASP.NET Core

ASP.NET Core có một hệ thống logging tích hợp mạnh mẽ và linh hoạt, hỗ trợ nhiều nhà cung cấp log (logging providers) khác nhau như Console, Debug, EventSource, EventLog, Azure App Services, và thậm chí là các thư viện logging của bên thứ ba như Serilog, NLog.

Các cấp độ Log (Log Levels):

ASP.NET Core định nghĩa các cấp độ log để phân loại mức độ nghiêm trọng của thông báo:

- **Trace (0):** Thông tin chi tiết nhất, thường chỉ dùng cho mục đích gỡ lỗi.
- **Debug (1):** Thông tin gỡ lỗi và hành vi của ứng dụng.
- **Information (2):** Thông tin chung về luồng hoạt động của ứng dụng.
- **Warning (3):** Một sự kiện bất thường hoặc không mong muốn xảy ra, nhưng ứng dụng vẫn có thể tiếp tục hoạt động.
- **Error (4):** Một lỗi đã xảy ra, làm gián đoạn một hoạt động hoặc chức năng.
- **Critical (5):** Một lỗi nghiêm trọng, có thể làm ứng dụng ngừng hoạt động hoặc yêu cầu sự can thiệp ngay lập tức.
- **None (6):** Không ghi log.

Cấu hình Logging:

Bạn có thể cấu hình các cấp độ log và nhà cung cấp log trong tệp `appsettings.json`:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "MyWebApp": "Debug" // Cấu hình cấp độ Debug cho namespace
của ứng dụng bạn
    },
    "Console": {
      "LogLevel": {
        "Default": "Information"
      }
    },
    "Debug": {
      "LogLevel": {
        "Default": "Information"
      }
    }
  },
  "AllowedHosts": "*"
}
```

Sử dụng `ILogger<T>`:

Bạn có thể inject `ILogger<T>` vào các lớp của mình thông qua Dependency Injection để ghi log. `T` thường là tên của lớp mà bạn đang ghi log.

```
// Controllers/HomeController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Logging;

namespace MyWebApp.Controllers
{
    public class HomeController : Controller
    {
        private readonly ILogger<HomeController> _logger;

        public HomeController(ILogger<HomeController> logger)
        {
            _logger = logger;
        }

        public IActionResult Index()
        {
            _logger.LogInformation("Người dùng đã truy cập trang chủ.");
            _logger.LogDebug("Dữ liệu trang chủ đang được tải...");

            try
            {
                // Giả lập một lỗi
                int zero = 0;
                int result = 10 / zero;
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Đã xảy ra lỗi khi xử lý trang chủ.");
            }

            return View();
        }
    }
}
```

Ghi log với các thư viện bên thứ ba (ví dụ: Serilog):

Serilog là một thư viện logging phổ biến, cung cấp nhiều tính năng mạnh mẽ hơn so với logging tích hợp sẵn của ASP.NET Core, bao gồm khả năng ghi log có cấu trúc (structured logging) và nhiều sink (nơi lưu trữ log) khác nhau.

Bước 1: Cài đặt gói NuGet:

```
dotnet add package Serilog.AspNetCore
dotnet add package Serilog.Sinks.Console
dotnet add package Serilog.Sinks.File
```

Bước 2: Cấu hình Serilog trong Program.cs:

```
// Program.cs
using Serilog;

var builder = WebApplication.CreateBuilder(args);

// Cấu hình Serilog
builder.Host.UseSerilog((context, services, configuration) =>
configuration
    .ReadFrom.Configuration(context.Configuration)
    .ReadFrom.Services(services)
    .Enrich.FromLogContext()
    .WriteTo.Console()
    .WriteTo.File("logs/log-.txt", rollingInterval:
RollingInterval.Day));

// ... (Add services)

var app = builder.Build();

// ... (Configure middleware)

app.Run();
```

Với cấu hình này, log sẽ được ghi ra console và vào các tệp log hàng ngày trong thư mục `logs/`.

2.6.2 Monitoring

Monitoring là việc thu thập và phân tích dữ liệu về hiệu suất và hành vi của ứng dụng để đảm bảo nó hoạt động như mong đợi và phát hiện sớm các vấn đề. Các công cụ monitoring có thể theo dõi CPU, bộ nhớ, số lượng request, thời gian phản hồi, lỗi, v.v.

Các phương pháp Monitoring cơ bản:

1. **Application Insights (Azure):** Một dịch vụ giám sát ứng dụng toàn diện của Azure, cung cấp khả năng theo dõi hiệu suất, lỗi, sử dụng, và các dependency. Nó tích hợp sâu với ASP.NET Core.

Cài đặt:

```
dotnet add package Microsoft.ApplicationInsights.AspNetCore
```

Cấu hình trong Program.cs:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddApplicationInsightsTelemetry(); // Thêm
dịch vụ Application Insights

// ...
```

Bạn cần cấu hình `InstrumentationKey` hoặc `ConnectionString` trong `appsettings.json` hoặc biến môi trường.

2. **Health Checks:** ASP.NET Core cung cấp một tính năng Health Checks cho phép bạn kiểm tra trạng thái sức khỏe của các thành phần trong ứng dụng (ví dụ: kết nối cơ sở dữ liệu, dịch vụ bên ngoài).

Cấu hình trong Program.cs:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddHealthChecks(); // Thêm dịch vụ Health
Checks
builder.Services.AddHealthChecks()
```

```

        .AddDbContextCheck<ApplicationDbContext>(); // Kiểm tra
        kết nối DbContext

        // ...

        var app = builder.Build();

        // ...

        app.MapHealthChecks("/health"); // Ánh xạ endpoint Health
        Checks

        app.Run();

```

Bạn có thể truy cập `/health` để kiểm tra trạng thái sức khỏe của ứng dụng.

3. **Metrics (Số liệu):** Thu thập các số liệu về hiệu suất (ví dụ: thời gian phản hồi, số lượng lỗi, CPU usage). Bạn có thể sử dụng các thư viện như Prometheus Client for .NET để xuất các số liệu này và visualize chúng bằng Grafana.

Logging và Monitoring là những công cụ không thể thiếu để duy trì và cải thiện chất lượng của ứng dụng theo thời gian. Việc thiết lập một hệ thống logging và monitoring hiệu quả sẽ giúp bạn nhanh chóng phát hiện và khắc phục sự cố, đồng thời hiểu rõ hơn về cách ứng dụng của bạn đang hoạt động trong môi trường thực tế.

2.7 Unit Testing

Unit Testing (Kiểm thử đơn vị) là một phương pháp kiểm thử phần mềm trong đó các đơn vị nhỏ nhất của code (ví dụ: phương thức, lớp) được kiểm thử một cách độc lập để xác minh rằng chúng hoạt động đúng như mong đợi. Unit testing là một phần quan trọng của quy trình phát triển phần mềm hiện đại, giúp đảm bảo chất lượng code, phát hiện lỗi sớm và tạo điều kiện thuận lợi cho việc tái cấu trúc (refactoring).

Lợi ích của Unit Testing:

- **Phát hiện lỗi sớm:** Lỗi được tìm thấy ở giai đoạn phát triển, giúp giảm chi phí sửa lỗi.

- **Cải thiện chất lượng code:** Buộc nhà phát triển phải viết code có cấu trúc tốt, dễ kiểm thử (ví dụ: sử dụng Dependency Injection).
- **Tài liệu sống:** Các unit test có thể đóng vai trò như tài liệu về cách các đơn vị code hoạt động.
- **Tăng cường tự tin khi refactor:** Khi có một bộ unit test tốt, bạn có thể tự tin thay đổi code mà không sợ làm hỏng các chức năng hiện có.
- **Tăng tốc độ phát triển:** Mặc dù ban đầu có vẻ tốn thời gian, nhưng về lâu dài, unit testing giúp giảm thời gian gỡ lỗi và tăng tốc độ phát triển tổng thể.

Các Framework Unit Testing phổ biến trong .NET:

- **xUnit.net:** Một framework kiểm thử đơn vị miễn phí, mã nguồn mở, hướng cộng đồng cho .NET. Đây là lựa chọn phổ biến và được khuyến nghị bởi Microsoft.
- **NUnit:** Một framework kiểm thử đơn vị lâu đời và mạnh mẽ cho .NET.
- **MSTest:** Framework kiểm thử của Microsoft, tích hợp chặt chẽ với Visual Studio.

Trong phần này, chúng ta sẽ sử dụng **xUnit.net** và thư viện **Moq** (một thư viện mocking) để minh họa.

Bước 1: Tạo Project Test

Trong cùng solution với dự án ASP.NET Core của bạn, tạo một project thư viện lớp (class library) mới cho các unit test. Ví dụ, nếu dự án của bạn là `MyWebApp`, bạn có thể tạo project test là `MyWebApp.Tests`.

```
dotnet new xunit -n MyWebApp.Tests
dotnet add MyWebApp.Tests reference MyWebApp
```

Bước 2: Cài đặt các gói NuGet cần thiết

Trong project `MyWebApp.Tests`, cài đặt các gói sau:

```
dotnet add MyWebApp.Tests package Moq
dotnet add MyWebApp.Tests package Microsoft.AspNetCore.Mvc.Testing
// Nếu kiểm thử Controller
```

- `xunit`: Framework kiểm thử.

- `Moq`: Thư viện mocking, dùng để tạo các đối tượng giả (mock objects) cho các dependency.
- `Microsoft.AspNetCore.Mvc.Testing`: Hỗ trợ kiểm thử tích hợp (integration testing) cho các ứng dụng ASP.NET Core, nhưng cũng hữu ích cho một số kịch bản unit test liên quan đến Controller.

Bước 3: Viết Unit Test

Giả sử chúng ta có một `ProductService` để quản lý logic nghiệp vụ liên quan đến sản phẩm:

`Services/ProductService.cs` (trong project `MyWebApp`):

```
using MyWebApp.Data;
using MyWebApp.Models;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;

namespace MyWebApp.Services
{
    public interface IProductService
    {
        Task<IEnumerable<Product>> GetAllProductsAsync();
        Task<Product> GetProductByIdAsync(int id);
        Task<Product> AddProductAsync(Product product);
        Task<bool> UpdateProductAsync(Product product);
        Task<bool> DeleteProductAsync(int id);
    }

    public class ProductService : IProductService
    {
        private readonly ApplicationDbContext _context;

        public ProductService(ApplicationDbContext context)
        {
            _context = context;
        }
    }
}
```

```

        public async Task<IEnumerable<Product>>
GetAllProductsAsync()
    {
        return await _context.Products.ToListAsync();
    }

    public async Task<Product> GetProductByIdAsync(int id)
    {
        return await _context.Products.FindAsync(id);
    }

    public async Task<Product> AddProductAsync(Product product)
    {
        _context.Products.Add(product);
        await _context.SaveChangesAsync();
        return product;
    }

    public async Task<bool> UpdateProductAsync(Product product)
    {
        _context.Entry(product).State = EntityState.Modified;
        try
        {
            await _context.SaveChangesAsync();
            return true;
        }
        catch (DbUpdateConcurrencyException)
        {
            if (!ProductExists(product.Id))
            {
                return false;
            }
            else
            {
                throw;
            }
        }
    }

    public async Task<bool> DeleteProductAsync(int id)

```

```

    {
        var product = await _context.Products.FindAsync(id);
        if (product == null)
        {
            return false;
        }

        _context.Products.Remove(product);
        await _context.SaveChangesAsync();
        return true;
    }

    private bool ProductExists(int id)
    {
        return _context.Products.Any(e => e.Id == id);
    }
}

```

Bây giờ, chúng ta sẽ viết unit test cho `ProductService`.

MyWebApp.Tests/ProductServiceTests.cs:

```

using Xunit;
using Moq;
using MyWebApp.Data;
using MyWebApp.Models;
using MyWebApp.Services;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;

namespace MyWebApp.Tests
{
    public class ProductServiceTests
    {
        // Phương thức trợ giúp để tạo DbContextOptions cho in-
        memory database
    }
}

```

```

        private DbContextOptions<ApplicationDbContext>
CreateNewContextOptions()
    {
        return new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(databaseName:
Guid.NewGuid().ToString()) // Sử dụng tên database duy nhất cho mỗi
test
        .Options;
    }

[Fact] // Đánh dấu đây là một test method
public async Task GetAllProductsAsync_ReturnsAllProducts()
{
    // Arrange (Thiết lập môi trường)
    var options = CreateNewContextOptions();
    using (var context = new ApplicationDbContext(options))
    {
        context.Products.Add(new Product { Id = 1, Name =
"Product A", Price = 10.00m, Stock = 100 });
        context.Products.Add(new Product { Id = 2, Name =
"Product B", Price = 20.00m, Stock = 200 });
        await context.SaveChangesAsync();
    }

    using (var context = new ApplicationDbContext(options))
    {
        var service = new ProductService(context);

        // Act (Thực hiện hành động)
        var result = await service.GetAllProductsAsync();

        // Assert (Kiểm tra kết quả)
        Assert.NotNull(result);
        Assert.Equal(2, result.Count());
        Assert.Contains(result, p => p.Name == "Product
A");
    }
}

```

```

[Fact]
public async Task
GetProductByIdAsync_ReturnsCorrectProduct()
{
    // Arrange
    var options = CreateNewContextOptions();
    using (var context = new ApplicationDbContext(options))
    {
        context.Products.Add(new Product { Id = 1, Name =
"Product A", Price = 10.00m, Stock = 100 });
        await context.SaveChangesAsync();
    }

    using (var context = new ApplicationDbContext(options))
    {
        var service = new ProductService(context);

        // Act
        var result = await service.GetProductByIdAsync(1);

        // Assert
        Assert.NotNull(result);
        Assert.Equal("Product A", result.Name);
    }
}

[Fact]
public async Task AddProductAsync_AddsProductToDatabase()
{
    // Arrange
    var options = CreateNewContextOptions();
    using (var context = new ApplicationDbContext(options))
    {
        var service = new ProductService(context);
        var newProduct = new Product { Id = 3, Name =
"Product C", Price = 30.00m, Stock = 300 };

        // Act
        var result = await
service.AddProductAsync(newProduct);

```

```

        // Assert
        Assert.NotNull(result);
        Assert.Equal("Product C", result.Name);
        Assert.Equal(1, await
context.Products.CountAsync());
    }
}

[Fact]
public async Task
DeleteProductAsync_RemovesProductFromDatabase()
{
    // Arrange
    var options = CreateNewContextOptions();
    using (var context = new ApplicationDbContext(options))
    {
        context.Products.Add(new Product { Id = 1, Name =
"Product A", Price = 10.00m, Stock = 100 });
        await context.SaveChangesAsync();
    }

    using (var context = new ApplicationDbContext(options))
    {
        var service = new ProductService(context);

        // Act
        var result = await service.DeleteProductAsync(1);

        // Assert
        Assert.True(result);
        Assert.Equal(0, await
context.Products.CountAsync());
    }
}

// Ví dụ sử dụng Moq cho các dependency phức tạp hơn
[Fact]
public async Task
GetProductByIdAsync_WithMockedDependency_ReturnsProduct()

```

```

{
    // Arrange
    var mockProduct = new Product { Id = 1, Name = "Mocked
Product", Price = 10.00m, Stock = 100 };

    // Tạo một mock cho DbSet<Product>
    var mockSet = new Mock<DbSet<Product>>();
    mockSet.As<IQueryable<Product>>().Setup(m =>
m.Provider).Returns(new List<Product> { mockProduct
}.AsQueryable().Provider);
    mockSet.As<IQueryable<Product>>().Setup(m =>
m.Expression).Returns(new List<Product> { mockProduct
}.AsQueryable().Expression);
    mockSet.As<IQueryable<Product>>().Setup(m =>
m.ElementType).Returns(new List<Product> { mockProduct
}.AsQueryable().ElementType);
    mockSet.As<IQueryable<Product>>().Setup(m =>
m.GetEnumerator()).Returns(new List<Product> { mockProduct
}.AsQueryable().GetEnumerator());
    mockSet.Setup(m => m.FindAsync(It.IsAny<object[]>()))
        .ReturnsAsync((object[] ids) => new
List<Product> { mockProduct }.FirstOrDefault(p => p.Id ==
(int)ids[0]));

    // Tạo một mock cho ApplicationDbContext
    var mockContext = new Mock<ApplicationDbContext>();
    mockContext.Setup(c =>
c.Products).Returns(mockSet.Object);

    var service = new ProductService(mockContext.Object);

    // Act
    var result = await service.GetProductByIdAsync(1);

    // Assert
    Assert.NotNull(result);
    Assert.Equal("Mocked Product", result.Name);
}
}
}

```


Giải thích:

- **[Fact]**: Thuộc tính này đánh dấu một phương thức là một test method trong xUnit.
- **Arrange-Act-Assert (AAA) Pattern**: Đây là một mẫu phổ biến để cấu trúc các unit test:
 - **Arrange**: Thiết lập tất cả các đối tượng, biến, và điều kiện cần thiết cho test.
 - **Act**: Thực hiện hành động mà bạn muốn kiểm thử (gọi phương thức, thực hiện một thao tác).
 - **Assert**: Xác minh rằng kết quả của hành động là đúng như mong đợi.
- **In-memory Database**: Đối với các test liên quan đến `DbContext`, việc sử dụng in-memory database (như `UseInMemoryDatabase`) là một cách tuyệt vời để kiểm thử logic tương tác với cơ sở dữ liệu mà không cần kết nối đến một cơ sở dữ liệu thực. Điều này giúp test chạy nhanh và độc lập.
- **Moq**: Khi một lớp có các dependency phức tạp (ví dụ: `ApplicationDbContext` với các `DbSet`), bạn có thể sử dụng Moq để tạo các đối tượng giả (mock objects) cho các dependency đó. Điều này cho phép bạn kiểm soát hành vi của các dependency và tập trung kiểm thử logic của lớp đang được test, mà không cần phải thiết lập toàn bộ môi trường thực.

Chạy Unit Test:

Bạn có thể chạy các unit test từ Command Line hoặc từ Visual Studio.

- **Command Line:**

Mở Terminal trong thư mục gốc của solution và chạy:

```
dotnet test
```

- **Visual Studio:**

Mở Test Explorer (Test > Test Explorer), các test của bạn sẽ xuất hiện ở đó. Bạn có thể chạy tất cả các test hoặc từng test riêng lẻ.

Unit testing là một kỹ năng quan trọng mà mọi nhà phát triển Middle Level nên thành thạo để xây dựng các ứng dụng chất lượng cao và đáng tin cậy.

3. Senior Level: Tối ưu hóa và bảo mật

Ở cấp độ Senior, bạn sẽ tập trung vào việc tối ưu hóa hiệu suất, bảo mật ứng dụng, và các kỹ thuật phát triển nâng cao. Đây là những kỹ năng cần thiết để xây dựng các ứng dụng doanh nghiệp có thể mở rộng và đáng tin cậy.

3.1 Performance Optimization (Caching, Asynchronous Programming)

Hiệu suất là một yếu tố quan trọng quyết định trải nghiệm người dùng và khả năng mở rộng của ứng dụng. ASP.NET Core cung cấp nhiều kỹ thuật để tối ưu hóa hiệu suất, trong đó hai kỹ thuật quan trọng nhất là **Caching** và **Asynchronous Programming**.

3.1.1 Caching (Bộ nhớ đệm)

Caching là kỹ thuật lưu trữ dữ liệu thường xuyên được truy cập vào một vị trí có thể truy cập nhanh hơn (ví dụ: bộ nhớ RAM) để giảm thời gian phản hồi và tải cho hệ thống. ASP.NET Core hỗ trợ nhiều loại caching khác nhau.

In-Memory Caching:

In-Memory Caching lưu trữ dữ liệu trong bộ nhớ của ứng dụng. Đây là loại cache nhanh nhất nhưng chỉ có thể truy cập trong cùng một instance của ứng dụng.

Cấu hình và sử dụng:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// Đăng ký dịch vụ Memory Cache
builder.Services.AddMemoryCache();

builder.Services.AddControllersWithViews();

var app = builder.Build();

// ... (Configure middleware)

app.Run();
```

Sử dụng trong Controller:

```
// Controllers/ProductsController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.Extensions.Caching.Memory;
using MyWebApp.Data;
using MyWebApp.Models;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;

namespace MyWebApp.Controllers
{
    public class ProductsController : Controller
    {
        private readonly ApplicationDbContext _context;
        private readonly IMemoryCache _cache;
        private const string PRODUCTS_CACHE_KEY = "products_list";

        public ProductsController(ApplicationDbContext context,
IMemoryCache cache)
        {
            _context = context;
            _cache = cache;
        }

        public async Task<IActionResult> Index()
        {
            // Kiểm tra xem dữ liệu đã có trong cache chưa
            if (!_cache.TryGetValue(PRODUCTS_CACHE_KEY, out
List<Product> products))
            {
                // Nếu chưa có, truy vấn từ database
                products = await _context.Products.ToListAsync();

                // Lưu vào cache với thời gian hết hạn 5 phút
                var cacheOptions = new MemoryCacheEntryOptions
                {
```

```

        AbsoluteExpirationRelativeToNow =
    TimeSpan.FromMinutes(5),
        SlidingExpiration = TimeSpan.FromMinutes(2), //
    Gia hạn thêm 2 phút nếu được truy cập
        Priority = CacheItemPriority.Normal
    };

    _cache.Set(PRODUCTS_CACHE_KEY, products,
cacheOptions);
    }

    return View(products);
}

// Xóa cache khi có thay đổi dữ liệu
[HttpPost]
public async Task<IActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();

        // Xóa cache để đảm bảo dữ liệu mới được tải lại
        _cache.Remove(PRODUCTS_CACHE_KEY);

        return RedirectToAction(nameof(Index));
    }
    return View(product);
}
}
}

```

Distributed Caching:

Distributed Caching cho phép chia sẻ cache giữa nhiều instance của ứng dụng. ASP.NET Core hỗ trợ nhiều nhà cung cấp distributed cache như Redis, SQL Server, NCache.

Cấu hình Redis Cache:

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

// Cấu hình Redis Distributed Cache
builder.Services.AddStackExchangeRedisCache(options =>
{
    options.Configuration = "localhost:6379"; // Chuỗi kết nối
    Redis
});

builder.Services.AddControllersWithViews();

var app = builder.Build();

// ...

app.Run();
```

Sử dụng Distributed Cache:

```
// Controllers/ProductsController.cs
using Microsoft.Extensions.Caching.Distributed;
using Newtonsoft.Json;

public class ProductsController : Controller
{
    private readonly ApplicationDbContext _context;
    private readonly IDistributedCache _distributedCache;
    private const string PRODUCTS_CACHE_KEY = "products_list";

    public ProductsController(ApplicationDbContext context,
        IDistributedCache distributedCache)
    {
        _context = context;
        _distributedCache = distributedCache;
    }

    public async Task<IActionResult> Index()
    {
```

```

        // Kiểm tra cache
        var cachedProducts = await
_distributedCache.GetStringAsync(PRODUCTS_CACHE_KEY);
        List<Product> products;

        if (cachedProducts != null)
        {
            // Deserialize từ JSON
            products = JsonConvert.DeserializeObject<List<Product>>
(cachedProducts);
        }
        else
        {
            // Truy vấn từ database
            products = await _context.Products.ToListAsync();

            // Serialize và lưu vào cache
            var serializedProducts =
JsonConvert.SerializeObject(products);
            var options = new DistributedCacheEntryOptions
            {
                AbsoluteExpirationRelativeToNow =
TimeSpan.FromMinutes(5),
                SlidingExpiration = TimeSpan.FromMinutes(2)
            };

            await
_distributedCache.SetStringAsync(PRODUCTS_CACHE_KEY,
serializedProducts, options);
        }

        return View(products);
    }
}

```

Response Caching:

Response Caching lưu trữ toàn bộ HTTP response để tránh phải xử lý lại các yêu cầu giống nhau.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddResponseCaching();

var app = builder.Build();

app.UseResponseCaching();

// ...
```

Sử dụng trong Controller:

```
[ResponseCache(Duration = 300)] // Cache trong 5 phút
public async Task<IActionResult> Index()
{
    var products = await _context.Products.ToListAsync();
    return View(products);
}

[ResponseCache(Duration = 0, Location = ResponseCacheLocation.None,
NoStore = true)]
public IActionResult Privacy()
{
    return View();
}
```

3.1.2 Asynchronous Programming (Lập trình bất đồng bộ)

Lập trình bất đồng bộ cho phép ứng dụng thực hiện các tác vụ khác trong khi chờ đợi các thao tác I/O (như truy cập database, gọi API bên ngoài) hoàn thành. Điều này giúp tăng khả năng xử lý đồng thời và cải thiện hiệu suất tổng thể của ứng dụng.

Từ khóa `async` và `await`:

```
// Phương thức đồng bộ (synchronous) - KHÔNG nên dùng cho I/O
operations
public IActionResult GetProducts()
{
    var products = _context.Products.ToList(); // Blocking call
    return View(products);
}

// Phương thức bất đồng bộ (asynchronous) - NÊN dùng
public async Task<IActionResult> GetProductsAsync()
{
    var products = await _context.Products.ToListAsync(); // Non-
blocking call
    return View(products);
}
```

Các nguyên tắc quan trọng:

1. Sử dụng `async/await` cho tất cả I/O operations: Database queries, HTTP calls, file operations.
2. Không sử dụng `.Result` hoặc `.Wait()`: Có thể gây deadlock.
3. Sử dụng `ConfigureAwait(false)` trong libraries: Để tránh deadlock trong một số trường hợp.

Ví dụ về HTTP Client bất đồng bộ:

```
// Services/ExternalApiService.cs
public class ExternalApiService
{
    private readonly HttpClient _httpClient;

    public ExternalApiService(HttpClient httpClient)
    {
        _httpClient = httpClient;
    }

    public async Task<string> GetDataFromExternalApiAsync(string
endpoint)
    {

```



```

        try
        {
            var response = await _httpClient.GetAsync(endpoint);
            response.EnsureSuccessStatusCode();
            return await response.Content.ReadAsStringAsync();
        }
        catch (HttpRequestException ex)
        {
            // Log error
            throw new Exception($"Error calling external API: {ex.Message}", ex);
        }
    }
}

// Đăng ký HttpClient trong Program.cs
builder.Services.AddHttpClient<ExternalApiService>();

```

Parallel Processing với Task.WhenAll:

```

public async Task<IActionResult> GetMultipleDataAsync()
{
    // Thực hiện nhiều tác vụ bất đồng bộ song song
    var task1 = _productService.GetProductsAsync();
    var task2 = _categoryService.GetCategoriesAsync();
    var task3 = _orderService.GetRecentOrdersAsync();

    // Chờ tất cả các tác vụ hoàn thành
    await Task.WhenAll(task1, task2, task3);

    var viewModel = new DashboardViewModel
    {
        Products = await task1,
        Categories = await task2,
        RecentOrders = await task3
    };

    return View(viewModel);
}

```

Việc kết hợp caching và asynchronous programming sẽ giúp ứng dụng ASP.NET Core của bạn đạt được hiệu suất tối ưu, có thể xử lý nhiều yêu cầu đồng thời và cung cấp trải nghiệm người dùng mượt mà.

3.2 Security Best Practices (XSS, CSRF, SQL Injection)

Bảo mật là một khía cạnh quan trọng không thể bỏ qua trong phát triển ứng dụng web. ASP.NET Core cung cấp nhiều tính năng bảo mật tích hợp sẵn, nhưng nhà phát triển vẫn cần hiểu và áp dụng các thực hành tốt nhất để bảo vệ ứng dụng khỏi các mối đe dọa phổ biến.

3.2.1 Cross-Site Scripting (XSS)

XSS là một lỗ hổng bảo mật cho phép kẻ tấn công chèn mã JavaScript độc hại vào trang web, mã này sẽ được thực thi trong trình duyệt của người dùng khác. Điều này có thể dẫn đến việc đánh cắp cookie, session token, hoặc thực hiện các hành động thay mặt người dùng.

Các loại XSS:

1. **Stored XSS:** Mã độc hại được lưu trữ trên server (ví dụ: trong database) và được hiển thị cho tất cả người dùng truy cập trang.
2. **Reflected XSS:** Mã độc hại được phản ánh ngay lập tức trong phản hồi HTTP, thường thông qua URL parameters.
3. **DOM-based XSS:** Mã độc hại được thực thi thông qua việc thao tác DOM ở phía client.

Phòng chống XSS trong ASP.NET Core:

1. Automatic HTML Encoding:

ASP.NET Core tự động mã hóa (encode) HTML trong Razor Views, ngăn chặn việc thực thi mã JavaScript không mong muốn.

```
<!-- An toàn: ASP.NET Core sẽ tự động encode HTML -->
<p>Tên người dùng: @Model.UserName</p>

<!-- Nếu Model.UserName = "<script>alert('XSS')</script>",
     nó sẽ được hiển thị như text thay vì thực thi script -->
```

2. Sử dụng `Html.Raw()` một cách cẩn thận:

Chỉ sử dụng `Html.Raw()` khi bạn chắc chắn rằng nội dung đã được làm sạch (sanitized).

```
<!-- NGUY HIỂM: Không encode HTML -->
@Html.Raw(Model.HtmlContent)

<!-- AN TOÀN: Sử dụng thư viện sanitization -->
@Html.Raw(HtmlSanitizer.Sanitize(Model.HtmlContent))
```

3. Content Security Policy (CSP):

CSP là một header HTTP giúp ngăn chặn XSS bằng cách kiểm soát các nguồn tài nguyên mà trang web có thể tải.

```
// Program.cs hoặc trong middleware tùy chỉnh
app.Use(async (context, next) =>
{
    context.Response.Headers.Add("Content-Security-Policy",
        "default-src 'self'; script-src 'self' 'unsafe-inline';
        style-src 'self' 'unsafe-inline'");
    await next();
});
```

4. Validation và Sanitization:

```
// Models/CommentModel.cs
using System.ComponentModel.DataAnnotations;

public class CommentModel
{
    [Required]
    [StringLength(1000)]
    [RegularExpression(@"^[a-zA-Z0-9\s\.,!]*$", ErrorMessage =
        "Chỉ cho phép ký tự chữ, số và dấu câu cơ bản.")]
    public string Content { get; set; }
}

// Sử dụng thư viện HtmlSanitizer
```

```
// dotnet add package HtmlSanitizer
using Ganss.XSS;

public class CommentService
{
    private readonly HtmlSanitizer _htmlSanitizer;

    public CommentService()
    {
        _htmlSanitizer = new HtmlSanitizer();
        _htmlSanitizer.AllowedTags.Clear();
        _htmlSanitizer.AllowedTags.Add("b");
        _htmlSanitizer.AllowedTags.Add("i");
        _htmlSanitizer.AllowedTags.Add("em");
        _htmlSanitizer.AllowedTags.Add("strong");
    }

    public string SanitizeComment(string comment)
    {
        return _htmlSanitizer.Sanitize(comment);
    }
}
```

3.2.2 Cross-Site Request Forgery (CSRF)

CSRF là một cuộc tấn công buộc người dùng đã được xác thực thực hiện các hành động không mong muốn trên một ứng dụng web mà họ đang đăng nhập.

Phòng chống CSRF trong ASP.NET Core:

1. Anti-Forgery Tokens:

ASP.NET Core tự động tạo và xác thực anti-forgery tokens.

```
// Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAntiforgery(options =>
{

```

```

options.HeaderName = "X-CSRF-TOKEN";
options.Cookie.Name = "__RequestVerificationToken";
options.Cookie.HttpOnly = true;
options.Cookie.SecurePolicy = CookieSecurePolicy.Always; // Chỉ
gửi qua HTTPS
});

builder.Services.AddControllersWithViews();

var app = builder.Build();

// ...

```

2. Sử dụng `[ValidateAntiForgeryToken]` trong Controller:

```

// Controllers/ProductsController.cs
[HttpPost]
[ValidateAntiForgeryToken] // Xác thực anti-forgery token
public async Task<IActionResult> Create(Product product)
{
    if (ModelState.IsValid)
    {
        _context.Add(product);
        await _context.SaveChangesAsync();
        return RedirectToAction(nameof(Index));
    }
    return View(product);
}

```

3. Thêm token vào form trong Razor View:

```

<!-- Razor tự động thêm anti-forgery token khi sử dụng asp-action -
->
<form asp-action="Create" method="post">
    @Html.AntiForgeryToken() <!-- Hoặc thêm thủ công -->
    <div class="form-group">
        <label asp-for="Name"></label>
        <input asp-for="Name" class="form-control" />
    </div>
    <button type="submit">Tạo</button>
</form>

```

4. CSRF protection cho AJAX requests:

```

// Lấy token từ HTML meta tag
var token = $('input[name="__RequestVerificationToken"]').val();

$.ajaxSetup({
    beforeSend: function(xhr) {
        xhr.setRequestHeader('X-CSRF-TOKEN', token);
    }
});

// Hoặc thêm vào form data
$.post('/Products/Create', {
    __RequestVerificationToken: token,
    Name: 'Product Name',
    Price: 100
});

```

3.2.3 SQL Injection

SQL Injection là một kỹ thuật tấn công cho phép kẻ tấn công chèn mã SQL độc hại vào các truy vấn database, có thể dẫn đến việc truy cập trái phép, sửa đổi, hoặc xóa dữ liệu.

Phòng chống SQL Injection:

1. Sử dụng Entity Framework Core (ORM):

EF Core tự động tham số hóa (parameterize) các truy vấn, ngăn chặn SQL injection.

```
// AN TOÀN: EF Core tự động parameterize
var products = await _context.Products
    .Where(p => p.Name.Contains(searchTerm))
    .ToListAsync();

// AN TOÀN: Sử dụng LINQ
var product = await _context.Products
    .FirstOrDefaultAsync(p => p.Id == productId);
```

2. Sử dụng Parameterized Queries khi cần Raw SQL:

```
// AN TOÀN: Sử dụng parameters
var products = await _context.Products
    .FromSqlRaw("SELECT * FROM Products WHERE Name LIKE {0}", $"%{searchTerm}%")
    .ToListAsync();

// NGUY HIỂM: String concatenation
// var products = await _context.Products
//     .FromSqlRaw($"SELECT * FROM Products WHERE Name LIKE '%{searchTerm}%'")
//     .ToListAsync();
```

3. Stored Procedures với Parameters:

```
// Sử dụng stored procedure với parameters
var products = await _context.Products
    .FromSqlRaw("EXEC GetProductsByCategory @CategoryId = {0}",
        categoryId)
    .ToListAsync();
```

4. Input Validation:

```
// Models/ProductSearchModel.cs
public class ProductSearchModel
{
    [Required]
    [StringLength(100)]
    [RegularExpression(@"^[a-zA-Z0-9\s]*$", ErrorMessage = "Chỉ cho phép ký tự chữ và số.")]
    public string SearchTerm { get; set; }

    [Range(1, int.MaxValue, ErrorMessage = "Category ID phải là số dương.")]
    public int? CategoryId { get; set; }
}
```

3.2.4 Các Security Headers quan trọng

```
// Middleware/SecurityHeadersMiddleware.cs
public class SecurityHeadersMiddleware
{
    private readonly RequestDelegate _next;

    public SecurityHeadersMiddleware(RequestDelegate next)
    {
        _next = next;
    }

    public async Task InvokeAsync(HttpContext context)
    {
        // X-Content-Type-Options: Ngăn chặn MIME type sniffing
        context.Response.Headers.Add("X-Content-Type-Options",
            "nosniff");

        // X-Frame-Options: Ngăn chặn clickjacking
        context.Response.Headers.Add("X-Frame-Options", "DENY");

        // X-XSS-Protection: Kích hoạt XSS filtering trong browser
        context.Response.Headers.Add("X-XSS-Protection", "1;
            mode=block");
    }
}
```



```

        // Strict-Transport-Security: Buộc sử dụng HTTPS
        context.Response.Headers.Add("Strict-Transport-Security",
            "max-age=31536000; includeSubDomains");

        // Referrer-Policy: Kiểm soát thông tin referrer
        context.Response.Headers.Add("Referrer-Policy", "strict-
            origin-when-cross-origin");

        await _next(context);
    }
}

// Đăng ký middleware trong Program.cs
app.UseMiddleware<SecurityHeadersMiddleware>();

```

3.2.5 HTTPS và Certificate Management

```

// Program.cs
var builder = WebApplication.CreateBuilder(args);

// ...

var app = builder.Build();

// Redirect HTTP to HTTPS
app.UseHttpsRedirection();

// HSTS (HTTP Strict Transport Security)
if (!app.Environment.IsDevelopment())
{
    app.UseHsts();
}

// ...

```

Cấu hình HTTPS trong appsettings.json:

```
{
```

```
"Kestrel": {
  "Endpoints": {
    "Http": {
      "Url": "http://localhost:5000"
    },
    "Https": {
      "Url": "https://localhost:5001",
      "Certificate": {
        "Path": "certificate.pfx",
        "Password": "your-certificate-password"
      }
    }
  }
}
```

Bảo mật là một quá trình liên tục và cần được tích hợp vào mọi giai đoạn của quá trình phát triển. Việc hiểu và áp dụng các thực hành bảo mật này sẽ giúp bảo vệ ứng dụng và dữ liệu của bạn khỏi các mối đe dọa phổ biến.

3.3 Advanced Authentication and Authorization (IdentityServer, JWT)

Ở cấp độ Senior, bạn cần hiểu sâu hơn về các cơ chế xác thực và ủy quyền nâng cao, đặc biệt là trong các kiến trúc phân tán và microservices. Hai công nghệ quan trọng cần nắm vững là **JSON Web Tokens (JWT)** và **IdentityServer**.

3.3.1 JSON Web Tokens (JWT)

JWT là một tiêu chuẩn mở (RFC 7519) để truyền thông tin an toàn giữa các bên dưới dạng một đối tượng JSON. JWT thường được sử dụng cho authentication và authorization trong các API và ứng dụng phân tán.

Cấu trúc của JWT:

JWT bao gồm ba phần được phân tách bởi dấu chấm (.):

1. **Header:** Chứa thông tin về thuật toán mã hóa và loại token.

2. **Payload:** Chứa các claims (thông tin về người dùng và metadata).
3. **Signature:** Được tạo bằng cách mã hóa header và payload với một secret key.

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Cấu hình JWT Authentication trong ASP.NET Core:

```
// Program.cs
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// Cấu hình JWT Authentication
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme =
JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme =
JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new
TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidateAudience = true,
        ValidateLifetime = true,
        ValidateIssuerSigningKey = true,
        ValidIssuer = builder.Configuration["Jwt:Issuer"],
        ValidAudience = builder.Configuration["Jwt:Audience"],
        IssuerSigningKey = new SymmetricSecurityKey(
Encoding.UTF8.GetBytes(builder.Configuration["Jwt:SecretKey"]))
    };
});
```

```
});

builder.Services.AddAuthorization();
builder.Services.AddControllers();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers();

app.Run();
```

Cấu hình trong appsettings.json:

```
{
  "Jwt": {
    "SecretKey": "your-super-secret-key-that-is-at-least-256-bits-long",
    "Issuer": "https://your-app.com",
    "Audience": "https://your-app.com",
    "ExpirationInMinutes": 60
  }
}
```

Tạo JWT Token Service:

```
// Services/JwtTokenService.cs
using Microsoft.IdentityModel.Tokens;
using System.IdentityModel.Tokens.Jwt;
using System.Security.Claims;
using System.Text;

public interface IJwtTokenService
{
    string GenerateToken(string userId, string username,
        IList<string> roles);
    ClaimsPrincipal ValidateToken(string token);
}
```

```

}

public class JwtTokenService : IJwtTokenService
{
    private readonly IConfiguration _configuration;

    public JwtTokenService(IConfiguration configuration)
    {
        _configuration = configuration;
    }

    public string GenerateToken(string userId, string username,
        IList<string> roles)
    {
        var claims = new List<Claim>
        {
            new Claim(ClaimTypes.NameIdentifier, userId),
            new Claim(ClaimTypes.Name, username),
            new Claim(JwtRegisteredClaimNames.Jti,
                Guid.NewGuid().ToString()),
            new Claim(JwtRegisteredClaimNames.Iat,
                new
                DateTimeOffset(DateTime.UtcNow).ToUnixTimeSeconds().ToString(),
                ClaimValueTypes.Integer64)
        };

        // Thêm roles vào claims
        foreach (var role in roles)
        {
            claims.Add(new Claim(ClaimTypes.Role, role));
        }

        var key = new
        SymmetricSecurityKey(Encoding.UTF8.GetBytes(_configuration["Jwt:Sec
        retKey"]));
        var credentials = new SigningCredentials(key,
        SecurityAlgorithms.HmacSha256);

        var token = new JwtSecurityToken(
            issuer: _configuration["Jwt:Issuer"],

```

```

        audience: _configuration["Jwt:Audience"],
        claims: claims,
        expires:
DateTime.UtcNow.AddMinutes(Convert.ToDouble(_configuration["Jwt:Exp
irationInMinutes"])),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}

public ClaimsPrincipal ValidateToken(string token)
{
    var tokenHandler = new JwtSecurityTokenHandler();
    var key =
Encoding.UTF8.GetBytes(_configuration["Jwt:SecretKey"]);

    try
    {
        var principal = tokenHandler.ValidateToken(token, new
TokenValidationParameters
        {
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(key),
            ValidateIssuer = true,
            ValidIssuer = _configuration["Jwt:Issuer"],
            ValidateAudience = true,
            ValidAudience = _configuration["Jwt:Audience"],
            ValidateLifetime = true,
            ClockSkew = TimeSpan.Zero
        }, out SecurityToken validatedToken);

        return principal;
    }
    catch
    {
        return null;
    }
}
}

```

Authentication Controller:

```
// Controllers/AuthController.cs
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Identity;

[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IJwtTokenService _jwtTokenService;
    private readonly UserManager<IdentityUser> _userManager;
    private readonly SignInManager<IdentityUser> _signInManager;

    public AuthController(
        IJwtTokenService jwtTokenService,
        UserManager<IdentityUser> userManager,
        SignInManager<IdentityUser> signInManager)
    {
        _jwtTokenService = jwtTokenService;
        _userManager = userManager;
        _signInManager = signInManager;
    }

    [HttpPost("login")]
    public async Task<IActionResult> Login([FromBody] LoginModel
model)
    {
        var user = await
_userManager.FindByNameAsync(model.Username);
        if (user != null && await
_userManager.CheckPasswordAsync(user, model.Password))
        {
            var roles = await _userManager.GetRolesAsync(user);
            var token = _jwtTokenService.GenerateToken(user.Id,
user.UserName, roles);

            return Ok(new
            {
                token = token,
            }
        )
    }
}
```

```

        expiration = DateTime.UtcNow.AddMinutes(60),
        user = new
        {
            id = user.Id,
            username = user.UserName,
            email = user.Email,
            roles = roles
        }
    });
}

return Unauthorized(new { message = "Invalid username or
password" });
}

[HttpPost("refresh")]
public async Task<IActionResult> RefreshToken([FromBody]
RefreshTokenModel model)
{
    var principal =
_jwtTokenService.ValidateToken(model.Token);
    if (principal == null)
    {
        return Unauthorized(new { message = "Invalid token" });
    }

    var userId =
principal.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var user = await _userManager.FindByIdAsync(userId);
    if (user == null)
    {
        return Unauthorized(new { message = "User not found"
});
    }

    var roles = await _userManager.GetRolesAsync(user);
    var newToken = _jwtTokenService.GenerateToken(user.Id,
user.UserName, roles);

    return Ok(new { token = newToken });
}

```



```

    }
}

public class LoginModel
{
    public string Username { get; set; }
    public string Password { get; set; }
}

public class RefreshTokenModel
{
    public string Token { get; set; }
}

```

3.3.2 IdentityServer

IdentityServer là một framework mã nguồn mở cho ASP.NET Core, triển khai các giao thức OpenID Connect và OAuth 2.0. Nó cho phép bạn xây dựng một dịch vụ xác thực tập trung (centralized authentication service) cho nhiều ứng dụng và API.

Các khái niệm chính:

- **Identity Provider (IdP):** Dịch vụ xác thực người dùng và cấp phát tokens.
- **Client:** Ứng dụng yêu cầu xác thực (web app, mobile app, SPA).
- **Resource:** API hoặc tài nguyên được bảo vệ.
- **Scope:** Phạm vi truy cập mà client yêu cầu.

Cài đặt IdentityServer:

```

dotnet add package Duende.IdentityServer
dotnet add package Duende.IdentityServer.AspNetIdentity

```

Cấu hình cơ bản:

```

// Program.cs (IdentityServer project)
using Duende.IdentityServer;
using Microsoft.AspNetCore.Identity;
using Microsoft.EntityFrameworkCore;

```

```

var builder = WebApplication.CreateBuilder(args);

// Cấu hình Entity Framework và Identity
builder.Services.AddDbContext<ApplicationDbContext>(options =>

    options.UseSqlServer(builder.Configuration.GetConnectionString("De
faultConnection")));

builder.Services.AddIdentity<IdentityUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

// Cấu hình IdentityServer
builder.Services.AddIdentityServer(options =>
{
    options.Events.RaiseErrorEvents = true;
    options.Events.RaiseInformationEvents = true;
    options.Events.RaiseFailureEvents = true;
    options.Events.RaiseSuccessEvents = true;
})
.AddInMemoryIdentityResources(Config.IdentityResources)
.AddInMemoryApiScopes(Config.ApiScopes)
.AddInMemoryClients(Config.Clients)
.AddAspNetIdentity<IdentityUser>()
.AddDeveloperSigningCredential(); // Chỉ dùng cho development

var app = builder.Build();

app.UseIdentityServer();

app.Run();

```

Cấu hình Resources và Clients:

```

// Config.cs
using Duende.IdentityServer.Models;
using Duende.IdentityServer;

```

```

public static class Config
{
    public static IEnumerable<IdentityResource> IdentityResources
=>
        new IdentityResource[]
        {
            new IdentityResources.OpenId(),
            new IdentityResources.Profile(),
            new IdentityResources.Email()
        };

    public static IEnumerable<ApiScope> ApiScopes =>
        new ApiScope[]
        {
            new ApiScope("api1", "My API"),
            new ApiScope("api2", "Another API")
        };

    public static IEnumerable<Client> Clients =>
        new Client[]
        {
            // Machine-to-machine client (Client Credentials)
            new Client
            {
                ClientId = "client",
                ClientSecrets = { new Secret("secret".Sha256()) },
                AllowedGrantTypes = GrantTypes.ClientCredentials,
                AllowedScopes = { "api1" }
            },

            // Interactive client (Authorization Code + PKCE)
            new Client
            {
                ClientId = "interactive",
                ClientSecrets = { new Secret("secret".Sha256()) },
                AllowedGrantTypes = GrantTypes.Code,
                RedirectUri = { "https://localhost:5002/signin-
oidc" },
                PostLogoutRedirectUri = {
                    "https://localhost:5002/signout-callback-oidc" },
            }
        }
    }
}

```

```

        AllowedScopes = new List<string>
        {
            IdentityServerConstants.StandardScopes.OpenId,
            IdentityServerConstants.StandardScopes.Profile,
            "api1"
        }
    },

    // SPA client (Authorization Code + PKCE)
    new Client
    {
        ClientId = "spa",
        RequireClientSecret = false,
        AllowedGrantTypes = GrantTypes.Code,
        RequirePkce = true,
        RedirectUri = { "http://localhost:3000/callback"
    },

        PostLogoutRedirectUri = { "http://localhost:3000"
    },

        AllowedCorsOrigins = { "http://localhost:3000" },
        AllowedScopes = new List<string>
        {
            IdentityServerConstants.StandardScopes.OpenId,
            IdentityServerConstants.StandardScopes.Profile,
            "api1"
        }
    }
};
}

```

Bảo vệ API với IdentityServer:

```

// Program.cs (API project)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthentication("Bearer")
    .AddJwtBearer("Bearer", options =>
    {

```

```

        options.Authority = "https://localhost:5001"; //
IdentityServer URL
        options.TokenValidationParameters = new
TokenValidationParameters
        {
            ValidateAudience = false
        };
    });

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("ApiScope", policy =>
    {
        policy.RequireAuthenticatedUser();
        policy.RequireClaim("scope", "api1");
    });
});

builder.Services.AddControllers();

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapControllers().RequireAuthorization("ApiScope");

app.Run();

```

Client Application sử dụng IdentityServer:

```

// Program.cs (Client MVC app)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddAuthentication(options =>
{
    options.DefaultScheme = "Cookies";
    options.DefaultChallengeScheme = "oidc";
})

```

```

.AddCookie("Cookies")
.AddOpenIdConnect("oidc", options =>
{
    options.Authority = "https://localhost:5001";
    options.ClientId = "interactive";
    options.ClientSecret = "secret";
    options.ResponseType = "code";
    options.Scope.Clear();
    options.Scope.Add("openid");
    options.Scope.Add("profile");
    options.Scope.Add("api1");
    options.SaveTokens = true;
});

var app = builder.Build();

app.UseAuthentication();
app.UseAuthorization();

app.MapDefaultControllerRoute().RequireAuthorization();

app.Run();

```

JWT và IdentityServer là những công nghệ quan trọng trong việc xây dựng các hệ thống xác thực và ủy quyền hiện đại, đặc biệt trong các kiến trúc microservices và ứng dụng phân tán. Việc nắm vững những công nghệ này sẽ giúp bạn xây dựng các hệ thống bảo mật, có thể mở rộng và dễ bảo trì.

3.4 Microservices Architecture (Basic Concepts)

Microservices Architecture là một phương pháp thiết kế phần mềm trong đó một ứng dụng lớn được chia thành nhiều dịch vụ nhỏ, độc lập, có thể triển khai và mở rộng riêng biệt. Mỗi microservice chịu trách nhiệm cho một chức năng nghiệp vụ cụ thể và giao tiếp với các dịch vụ khác thông qua các API được định nghĩa rõ ràng.

3.4.1 Đặc điểm của Microservices

Lợi ích:

- **Khả năng mở rộng độc lập:** Mỗi service có thể được mở rộng riêng biệt dựa trên nhu cầu.
- **Công nghệ đa dạng:** Các team có thể chọn công nghệ phù hợp nhất cho từng service.
- **Triển khai độc lập:** Có thể triển khai từng service mà không ảnh hưởng đến các service khác.
- **Khả năng chịu lỗi:** Lỗi trong một service không làm sập toàn bộ hệ thống.
- **Team autonomy:** Các team có thể làm việc độc lập trên các service của họ.

Thách thức:

- **Phức tạp về mạng:** Giao tiếp giữa các service qua network có thể chậm và không đáng tin cậy.
- **Data consistency:** Khó duy trì tính nhất quán dữ liệu giữa các service.
- **Monitoring và debugging:** Khó theo dõi và gỡ lỗi trong hệ thống phân tán.
- **Overhead:** Chi phí vận hành và quản lý nhiều service.

3.4.2 Thiết kế Microservices với ASP.NET Core

Ví dụ: E-commerce System

Chúng ta sẽ thiết kế một hệ thống e-commerce đơn giản với các microservices:

- **Product Service:** Quản lý sản phẩm
- **Order Service:** Quản lý đơn hàng
- **User Service:** Quản lý người dùng
- **API Gateway:** Điểm vào duy nhất cho client

Product Service:

```
// ProductService/Models/Product.cs
public class Product
{
```

```

    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
    public int Stock { get; set; }
    public string Category { get; set; }
}

// ProductService/Controllers/ProductsController.cs
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    private readonly IProductRepository _repository;
    private readonly ILogger<ProductsController> _logger;

    public ProductsController(IProductRepository repository,
        ILogger<ProductsController> logger)
    {
        _repository = repository;
        _logger = logger;
    }

    [HttpGet]
    public async Task<ActionResult<IEnumerable<Product>>>
        GetProducts()
    {
        _logger.LogInformation("Getting all products");
        var products = await _repository.GetAllAsync();
        return Ok(products);
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Product>> GetProduct(int id)
    {
        var product = await _repository.GetByIdAsync(id);
        if (product == null)
        {
            _logger.LogWarning("Product with id {ProductId} not
found", id);
            return NotFound();
        }
    }
}

```



```

        }
        return Ok(product);
    }

    [HttpPost]
    public async Task<ActionResult<Product>> CreateProduct(Product
product)
    {
        var createdProduct = await
_repository.CreateAsync(product);
        _logger.LogInformation("Created product with id
{ProductId}", createdProduct.Id);
        return CreatedAtAction(nameof(GetProduct), new { id =
createdProduct.Id }, createdProduct);
    }

    [HttpPut("{id}/stock")]
    public async Task<IActionResult> UpdateStock(int id, [FromBody]
int newStock)
    {
        var success = await _repository.UpdateStockAsync(id,
newStock);
        if (!success)
        {
            return NotFound();
        }

        _logger.LogInformation("Updated stock for product
{ProductId} to {Stock}", id, newStock);
        return NoContent();
    }
}

// ProductService/Program.cs
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<ProductDbContext>(options =>

```

```

    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));

builder.Services.AddScoped<IProductRepository, ProductRepository>
();

// Health checks
builder.Services.AddHealthChecks()
    .AddDbContextCheck<ProductDbContext>();

// Swagger
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();
app.MapHealthChecks("/health");

app.Run();

```

Order Service:

```

// OrderService/Models/Order.cs
public class Order
{
    public int Id { get; set; }
    public int UserId { get; set; }
    public DateTime OrderDate { get; set; }
    public OrderStatus Status { get; set; }
}

```

```

        public decimal TotalAmount { get; set; }
        public List<OrderItem> Items { get; set; } = new
List<OrderItem>();
    }

    public class OrderItem
    {
        public int Id { get; set; }
        public int ProductId { get; set; }
        public int Quantity { get; set; }
        public decimal Price { get; set; }
    }

    public enum OrderStatus
    {
        Pending,
        Confirmed,
        Shipped,
        Delivered,
        Cancelled
    }

    // OrderService/Services/IProductService.cs
    public interface IProductService
    {
        Task<Product> GetProductAsync(int productId);
        Task<bool> UpdateStockAsync(int productId, int quantity);
    }

    // OrderService/Services/ProductService.cs
    public class ProductService : IProductService
    {
        private readonly HttpClient _httpClient;
        private readonly ILogger<ProductService> _logger;

        public ProductService(HttpClient httpClient,
            ILogger<ProductService> logger)
        {
            _httpClient = httpClient;
            _logger = logger;
        }
    }

```

```

    }

    public async Task<Product> GetProductAsync(int productId)
    {
        try
        {
            var response = await
_httpClient.GetAsync($"api/products/{productId}");
            if (response.IsSuccessStatusCode)
            {
                var json = await
response.Content.ReadAsStringAsync();
                return JsonSerializer.Deserialize<Product>(json,
new JsonSerializerOptions
                {
                    PropertyNameCaseInsensitive = true
                });
            }
            return null;
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Error getting product
{ProductId}", productId);
            return null;
        }
    }

    public async Task<bool> UpdateStockAsync(int productId, int
quantity)
    {
        try
        {
            var content = new StringContent(quantity.ToString(),
Encoding.UTF8, "application/json");
            var response = await
_httpClient.PutAsync($"api/products/{productId}/stock", content);
            return response.IsSuccessStatusCode;
        }
        catch (Exception ex)

```

```

        {
            _logger.LogError(ex, "Error updating stock for product
{ProductId}", productId);
            return false;
        }
    }
}

```

// OrderService/Controllers/OrdersController.cs

```

[ApiController]
[Route("api/[controller]")]
public class OrdersController : ControllerBase
{
    private readonly IOrderRepository _orderRepository;
    private readonly IProductService _productService;
    private readonly ILogger<OrdersController> _logger;

    public OrdersController(
        IOrderRepository orderRepository,
        IProductService productService,
        ILogger<OrdersController> logger)
    {
        _orderRepository = orderRepository;
        _productService = productService;
        _logger = logger;
    }

    [HttpPost]
    public async Task<ActionResult<Order>>
CreateOrder(CreateOrderRequest request)
    {
        // Validate products and calculate total
        decimal totalAmount = 0;
        var orderItems = new List<OrderItem>();

        foreach (var item in request.Items)
        {
            var product = await
_productService.GetProductAsync(item.ProductId);
            if (product == null)

```

```

        {
            return BadRequest($"Product {item.ProductId} not
found");
        }

        if (product.Stock < item.Quantity)
        {
            return BadRequest($"Insufficient stock for product
{item.ProductId}");
        }

        orderItems.Add(new OrderItem
        {
            ProductId = item.ProductId,
            Quantity = item.Quantity,
            Price = product.Price
        });

        totalAmount += product.Price * item.Quantity;
    }

    // Create order
    var order = new Order
    {
        UserId = request.UserId,
        OrderDate = DateTime.UtcNow,
        Status = OrderStatus.Pending,
        TotalAmount = totalAmount,
        Items = orderItems
    };

    var createdOrder = await
_orderRepository.CreateAsync(order);

    // Update stock for each product
    foreach (var item in createdOrder.Items)
    {
        await _productService.UpdateStockAsync(item.ProductId,
-item.Quantity);
    }

```

```

        _logger.LogInformation("Created order {OrderId} for user {UserId}", createdOrder.Id, createdOrder.UserId);

        return CreatedAtAction(nameof(GetOrder), new { id = createdOrder.Id }, createdOrder);
    }

    [HttpGet("{id}")]
    public async Task<ActionResult<Order>> GetOrder(int id)
    {
        var order = await _orderRepository.GetByIdAsync(id);
        if (order == null)
        {
            return NotFound();
        }
        return Ok(order);
    }
}

public class CreateOrderRequest
{
    public int UserId { get; set; }
    public List<OrderItemRequest> Items { get; set; }
}

public class OrderItemRequest
{
    public int ProductId { get; set; }
    public int Quantity { get; set; }
}

```

3.4.3 Service Communication

HTTP Communication với Polly (Resilience Patterns):

```

// Cài đặt Polly
// dotnet add package Polly.Extensions.Http

```

```
// Program.cs trong Order Service
builder.Services.AddHttpClient<IProductService, ProductService>
(client =>
{
    client.BaseAddress = new
Uri(builder.Configuration["Services:ProductService"]);
    client.Timeout = TimeSpan.FromSeconds(30);
}))
.AddPolicyHandler(GetRetryPolicy())
.AddPolicyHandler(GetCircuitBreakerPolicy());

static IAsyncPolicy<HttpResponseMessage> GetRetryPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .WaitAndRetryAsync(
            retryCount: 3,
            sleepDurationProvider: retryAttempt =>
TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)),
            onRetry: (outcome, timespan, retryCount, context) =>
            {
                Console.WriteLine($"Retry {retryCount} after
{timespan}s");
            });
}

static IAsyncPolicy<HttpResponseMessage> GetCircuitBreakerPolicy()
{
    return HttpPolicyExtensions
        .HandleTransientHttpError()
        .CircuitBreakerAsync(
            handledEventsAllowedBeforeBreaking: 3,
            durationOfBreak: TimeSpan.FromSeconds(30),
            onBreak: (exception, duration) =>
            {
                Console.WriteLine($"Circuit breaker opened for
{duration}s");
            },
            onReset: () =>
            {

```



```
        Console.WriteLine("Circuit breaker reset");
    });
}
```

3.4.4 API Gateway với Ocelot

API Gateway là điểm vào duy nhất cho tất cả client requests, nó định tuyến requests đến các microservices phù hợp.

```
// Cài đặt Ocelot
// dotnet add package Ocelot

// Program.cs (API Gateway)
using Ocelot.DependencyInjection;
using Ocelot.Middleware;

var builder = WebApplication.CreateBuilder(args);

builder.Configuration.AddJsonFile("ocelot.json", optional: false,
reloadOnChange: true);
builder.Services.AddOcelot();

// CORS
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowAll", policy =>
    {
        policy.AllowAnyOrigin()
            .AllowAnyMethod()
            .AllowAnyHeader();
    });
});

var app = builder.Build();

app.UseCors("AllowAll");
await app.UseOcelot();

app.Run();
```

ocelot.json configuration:

```
{
  "Routes": [
    {
      "DownstreamPathTemplate": "/api/products/{everything}",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5001
        }
      ],
      "UpstreamPathTemplate": "/api/products/{everything}",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
    },
    {
      "DownstreamPathTemplate": "/api/orders/{everything}",
      "DownstreamScheme": "https",
      "DownstreamHostAndPorts": [
        {
          "Host": "localhost",
          "Port": 5002
        }
      ],
      "UpstreamPathTemplate": "/api/orders/{everything}",
      "UpstreamHttpMethod": [ "GET", "POST", "PUT", "DELETE" ]
    }
  ],
  "GlobalConfiguration": {
    "BaseUrl": "https://localhost:5000"
  }
}
```

3.4.5 Service Discovery và Configuration

Consul Integration:

```
// dotnet add package Consul
```

```

// Program.cs
builder.Services.AddSingleton<IConsulClient, ConsulClient>(p => new
ConsulClient(consulConfig =>
{
    var address = builder.Configuration["Consul:Host"];
    consulConfig.Address = new Uri(address);
}));

// Service Registration
var app = builder.Build();

var consulClient = app.Services.GetRequiredService<IConsulClient>
();
var lifetime =
app.Services.GetRequiredService<IHostApplicationLifetime>();

var registration = new AgentServiceRegistration()
{
    ID = $"productservice-{Environment.MachineName}",
    Name = "productservice",
    Address = "localhost",
    Port = 5001,
    Tags = new[] { "api", "product" },
    Check = new AgentServiceCheck()
    {
        HTTP = "https://localhost:5001/health",
        Timeout = TimeSpan.FromSeconds(10),
        Interval = TimeSpan.FromSeconds(30)
    }
};

await consulClient.Agent.ServiceRegister(registration);

lifetime.ApplicationStopping.Register(async () =>
{
    await consulClient.Agent.ServiceDeregister(registration.ID);
});

```

Microservices architecture mang lại nhiều lợi ích nhưng cũng đi kèm với độ phức tạp cao. Việc hiểu rõ các khái niệm cơ bản và biết cách triển khai chúng với ASP.NET Core sẽ giúp bạn xây dựng các hệ thống có thể mở rộng và bảo trì được.

3.5 Containerization (Docker)

Docker là một nền tảng containerization cho phép bạn đóng gói ứng dụng và tất cả các dependencies của nó vào một container nhẹ, có thể chạy trên bất kỳ môi trường nào có Docker. Điều này giải quyết vấn đề "works on my machine" và giúp đảm bảo tính nhất quán giữa các môi trường development, testing, và production.

3.5.1 Lợi ích của Docker

- **Portability:** Container có thể chạy trên bất kỳ hệ thống nào có Docker.
- **Consistency:** Đảm bảo ứng dụng chạy giống nhau trên mọi môi trường.
- **Isolation:** Các container được cách ly với nhau và với host system.
- **Efficiency:** Container nhẹ hơn virtual machines, khởi động nhanh hơn.
- **Scalability:** Dễ dàng scale up/down số lượng container instances.

3.5.2 Dockerizing ASP.NET Core Application

Bước 1: Tạo Dockerfile

Dockerfile là một text file chứa các instructions để build Docker image.

```
# Dockerfile
# Sử dụng official ASP.NET Core runtime image
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
WORKDIR /app
EXPOSE 80
EXPOSE 443

# Sử dụng SDK image để build ứng dụng
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src

# Copy project files và restore dependencies
```

```

COPY ["MyWebApp/MyWebApp.csproj", "MyWebApp/"]
RUN dotnet restore "MyWebApp/MyWebApp.csproj"

# Copy source code và build ứng dụng
COPY . .
WORKDIR "/src/MyWebApp"
RUN dotnet build "MyWebApp.csproj" -c Release -o /app/build

# Publish ứng dụng
FROM build AS publish
RUN dotnet publish "MyWebApp.csproj" -c Release -o /app/publish
/p:UseAppHost=false

# Final stage: copy published app vào runtime image
FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .
ENTRYPOINT ["dotnet", "MyWebApp.dll"]

```

Bước 2: Tạo .dockerignore

```

# .dockerignore
**/.dockerignore
**/.env
**/.git
**/.gitignore
**/.project
**/.settings
**/.toolstarget
**/.vs
**/.vscode
**/.idea
**/*.proj.user
**/*.dbmdl
**/*.jfm
**/azds.yaml
**/bin
**/charts
**/docker-compose*

```

```
**/Dockerfile*
**/node_modules
**/npm-debug.log
**/obj
**/secrets.dev.yaml
**/values.dev.yaml
LICENSE
README.md
```

Bước 3: Build và Run Docker Image

```
# Build Docker image
docker build -t mywebapp:latest .

# Run container
docker run -d -p 8080:80 --name mywebapp-container mywebapp:latest

# View running containers
docker ps

# View logs
docker logs mywebapp-container

# Stop container
docker stop mywebapp-container

# Remove container
docker rm mywebapp-container
```

3.5.3 Multi-stage Build Optimization

Multi-stage build giúp tối ưu kích thước image bằng cách tách biệt quá trình build và runtime.

```
# Optimized Dockerfile với multi-stage build
FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine AS base
WORKDIR /app
EXPOSE 80
```

```

# Create non-root user
RUN addgroup -g 1001 -S appgroup && \
    adduser -S appuser -u 1001 -G appgroup
USER appuser

FROM mcr.microsoft.com/dotnet/sdk:8.0-alpine AS build
WORKDIR /src

# Copy csproj và restore dependencies (layer caching)
COPY ["MyWebApp.csproj", "./"]
RUN dotnet restore "MyWebApp.csproj"

# Copy source code
COPY . .
RUN dotnet build "MyWebApp.csproj" -c Release -o /app/build

FROM build AS publish
RUN dotnet publish "MyWebApp.csproj" -c Release -o /app/publish \
    --no-restore \
    --no-build \
    /p:PublishReadyToRun=true

FROM base AS final
WORKDIR /app
COPY --from=publish /app/publish .

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --
retries=3 \
    CMD curl -f http://localhost/health || exit 1

ENTRYPOINT ["dotnet", "MyWebApp.dll"]

```

3.5.4 Docker Compose cho Multi-service Applications

Docker Compose cho phép định nghĩa và chạy multi-container applications.

```

# docker-compose.yml
version: '3.8'

```

```
services:
  # SQL Server Database
  sqlserver:
    image: mcr.microsoft.com/mssql/server:2022-latest
    container_name: sqlserver
    environment:
      - ACCEPT_EULA=Y
      - SA_PASSWORD=YourStrong@Passw0rd
      - MSSQL_PID=Express
    ports:
      - "1433:1433"
    volumes:
      - sqlserver_data:/var/opt/mssql
    networks:
      - app-network

  # Redis Cache
  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - "6379:6379"
    volumes:
      - redis_data:/data
    networks:
      - app-network

  # Web Application
  webapp:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: webapp
    ports:
      - "8080:80"
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
```



```

-
ConnectionStrings__DefaultConnection=Server=sqlserver;Database=MyWebAppDb;User
Id=sa;Password=YourStrong@Passw0rd;TrustServerCertificate=true
  - Redis__Configuration=redis:6379
  depends_on:
    - sqlserver
    - redis
  networks:
    - app-network
  restart: unless-stopped

# Nginx Reverse Proxy
nginx:
  image: nginx:alpine
  container_name: nginx
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
    - ./ssl:/etc/nginx/ssl
  depends_on:
    - webapp
  networks:
    - app-network

volumes:
  sqlserver_data:
  redis_data:

networks:
  app-network:
    driver: bridge

```

nginx.conf:

```

events {
    worker_connections 1024;

```

```

}

http {
    upstream webapp {
        server webapp:80;
    }

    server {
        listen 80;
        server_name localhost;

        location / {
            proxy_pass http://webapp;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
            proxy_set_header X-Forwarded-Proto $scheme;
        }

        location /health {
            proxy_pass http://webapp/health;
        }
    }
}

```

Chạy với Docker Compose:

```

# Start all services
docker-compose up -d

# View logs
docker-compose logs -f webapp

# Scale webapp service
docker-compose up -d --scale webapp=3

# Stop all services
docker-compose down

```

```
# Stop và remove volumes
docker-compose down -v
```

3.5.5 Environment-specific Configuration

```
# docker-compose.override.yml (Development)
version: '3.8'

services:
  webapp:
    environment:
      - ASPNETCORE_ENVIRONMENT=Development
      - ASPNETCORE_URLS=https://+:443;http://+:80
      -
    ASPNETCORE_Kestrel__Certificates__Default__Password=password
      -
    ASPNETCORE_Kestrel__Certificates__Default__Path=/https/aspnetapp.pfx
    x
    volumes:
      - ~/.aspnet/https:/https:ro
    ports:
      - "5000:80"
      - "5001:443"

# docker-compose.prod.yml (Production)
version: '3.8'

services:
  webapp:
    environment:
      - ASPNETCORE_ENVIRONMENT=Production
    deploy:
      replicas: 3
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 256M
```

3.5.6 Docker Best Practices cho ASP.NET Core

1. Security:

```
# Sử dụng non-root user
FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS base
RUN groupadd -r appgroup && useradd -r -g appgroup appuser
USER appuser

# Scan for vulnerabilities
# docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
#   -v $HOME/Library/Caches:/root/.cache/ \
#   aquasec/trivy image mywebapp:latest
```

2. Performance:

```
# Sử dụng Alpine images (nhỏ hơn)
FROM mcr.microsoft.com/dotnet/aspnet:8.0-alpine

# Enable ReadyToRun images
RUN dotnet publish -c Release -o /app/publish \
  /p:PublishReadyToRun=true \
  /p:PublishSingleFile=false
```

3. Monitoring:

```
# Health checks
HEALTHCHECK --interval=30s --timeout=10s --start-period=5s --
retries=3 \
  CMD curl -f http://localhost/health || exit 1
```

4. Configuration Management:

```
// Program.cs - Docker-friendly configuration
var builder = WebApplication.CreateBuilder(args);

// Read configuration from environment variables
builder.Configuration.AddEnvironmentVariables();
```

```
// Configure Kestrel for container
builder.WebHost.ConfigureKestrel(options =>
{
    options.ListenAnyIP(80);
    if (builder.Environment.IsProduction())
    {
        options.ListenAnyIP(443, listenOptions =>
        {
            listenOptions.UseHttps();
        });
    }
});
```

Docker giúp đơn giản hóa việc triển khai và quản lý ứng dụng ASP.NET Core, đặc biệt trong các môi trường microservices và cloud. Việc nắm vững Docker sẽ giúp bạn xây dựng các ứng dụng có thể portable, scalable và dễ bảo trì.

3.6 Deployment Strategies (Azure, AWS)

Triển khai ứng dụng ASP.NET Core lên cloud là một kỹ năng quan trọng. Hai nền tảng cloud phổ biến nhất là Microsoft Azure và Amazon Web Services (AWS). Mỗi nền tảng cung cấp nhiều tùy chọn triển khai khác nhau.

3.6.1 Azure Deployment

Azure App Service:

Azure App Service là một platform-as-a-service (PaaS) cho phép triển khai web apps mà không cần quản lý infrastructure.

```
# Azure CLI deployment
az login
az group create --name myResourceGroup --location "East US"
az appservice plan create --name myAppServicePlan --resource-group myResourceGroup --sku B1
az webapp create --resource-group myResourceGroup --plan myAppServicePlan --name myUniqueAppName --runtime "DOTNET|8.0"

# Deploy từ local
dotnet publish -c Release
cd bin/Release/net8.0/publish
zip -r ../myapp.zip .
az webapp deployment source config-zip --resource-group myResourceGroup --name myUniqueAppName --src ../myapp.zip
```

Azure Container Instances:

```
# Deploy container
az container create \
  --resource-group myResourceGroup \
  --name mycontainerapp \
  --image myregistry.azurecr.io/mywebapp:latest \
  --cpu 1 \
  --memory 1 \
  --ports 80 \
  --environment-variables ASPNETCORE_ENVIRONMENT=Production
```

3.6.2 AWS Deployment

AWS Elastic Beanstalk:

```
# Install EB CLI
pip install awsebcli

# Initialize và deploy
eb init
eb create production-env
eb deploy
```

Amazon ECS (Elastic Container Service):

```
{
  "family": "mywebapp-task",
  "networkMode": "awsvpc",
  "requiresCompatibilities": ["FARGATE"],
  "cpu": "256",
  "memory": "512",
  "executionRoleArn":
  "arn:aws:iam::account:role/ecsTaskExecutionRole",
  "containerDefinitions": [
    {
      "name": "mywebapp",
      "image":
      "myaccount.dkr.ecr.region.amazonaws.com/mywebapp:latest",
      "portMappings": [
        {
          "containerPort": 80,
          "protocol": "tcp"
        }
      ],
      "environment": [
        {
          "name": "ASPNETCORE_ENVIRONMENT",
          "value": "Production"
        }
      ],
      "logConfiguration": {
        "logDriver": "awslogs",
        "options": {
          "awslogs-group": "/ecs/mywebapp",
          "awslogs-region": "us-east-1",
          "awslogs-stream-prefix": "ecs"
        }
      }
    }
  ]
}
```

3.7 Integration Testing

Integration testing kiểm tra sự tương tác giữa các thành phần của ứng dụng, đảm bảo chúng hoạt động đúng khi kết hợp với nhau.

```
// IntegrationTests/CustomWebApplicationFactory.cs
public class CustomWebApplicationFactory<TStartup> :
    WebApplicationFactory<TStartup> where TStartup : class
{
    protected override void ConfigureWebHost(IWebHostBuilder
builder)
    {
        builder.ConfigureServices(services =>
        {
            // Remove existing DbContext
            var descriptor = services.SingleOrDefault(d =>
d.ServiceType == typeof(DbContextOptions<ApplicationDbContext>));
            if (descriptor != null)
                services.Remove(descriptor);

            // Add in-memory database
            services.AddDbContext<ApplicationDbContext>(options =>
            {
                options.UseInMemoryDatabase("InMemoryDbForTesting");
            });

            // Build service provider
            var sp = services.BuildServiceProvider();

            // Create scope và seed database
            using (var scope = sp.CreateScope())
            {
                var scopedServices = scope.ServiceProvider;
                var db =
scopedServices.GetRequiredService<ApplicationDbContext>();

                db.Database.EnsureCreated();
                SeedDatabase(db);
            }
        });
    }
}
```



```

    }

    private void SeedDatabase(ApplicationDbContext context)
    {
        context.Products.AddRange(
            new Product { Id = 1, Name = "Test Product 1", Price =
10.00m, Stock = 100 },
            new Product { Id = 2, Name = "Test Product 2", Price =
20.00m, Stock = 50 }
        );
        context.SaveChanges();
    }
}

// IntegrationTests/ProductsControllerTests.cs
public class ProductsControllerTests :
    IClassFixture<CustomWebApplicationFactory<Program>>
{
    private readonly CustomWebApplicationFactory<Program> _factory;
    private readonly HttpClient _client;

    public
    ProductsControllerTests(CustomWebApplicationFactory<Program>
factory)
    {
        _factory = factory;
        _client = _factory.CreateClient();
    }

    [Fact]
    public async Task
    GetProducts_ReturnsSuccessAndCorrectContentType()
    {
        // Act
        var response = await _client.GetAsync("/api/products");

        // Assert
        response.EnsureSuccessStatusCode();
        Assert.Equal("application/json; charset=utf-8",
response.Content.Headers.ContentType.ToString());
    }
}

```

```

        var jsonString = await
response.Content.ReadAsStringAsync();
        var products = JsonSerializer.Deserialize<List<Product>>
(jsonString, new JsonSerializerOptions
    {
        PropertyNameCaseInsensitive = true
    });

    Assert.Equal(2, products.Count);
}

[Fact]
public async Task CreateProduct_WithValidData_ReturnsCreated()
{
    // Arrange
    var newProduct = new Product
    {
        Name = "New Product",
        Price = 15.00m,
        Stock = 75
    };

    var json = JsonSerializer.Serialize(newProduct);
    var content = new StringContent(json, Encoding.UTF8,
"application/json");

    // Act
    var response = await _client.PostAsync("/api/products",
content);

    // Assert
    Assert.Equal(HttpStatusCode.Created, response.StatusCode);
}
}

```

3.8 Real-time Communication (SignalR)

SignalR cho phép thêm chức năng real-time vào ứng dụng web, cho phép server push content đến client ngay lập tức.

```
// Hubs/ChatHub.cs
public class ChatHub : Hub
{
    public async Task SendMessage(string user, string message)
    {
        await Clients.All.SendAsync("ReceiveMessage", user,
message);
    }

    public async Task JoinGroup(string groupName)
    {
        await Groups.AddToGroupAsync(Context.ConnectionId,
groupName);
        await Clients.Group(groupName).SendAsync("UserJoined",
Context.UserIdentifier);
    }

    public async Task LeaveGroup(string groupName)
    {
        await Groups.RemoveFromGroupAsync(Context.ConnectionId,
groupName);
        await Clients.Group(groupName).SendAsync("UserLeft",
Context.UserIdentifier);
    }

    public override async Task OnConnectedAsync()
    {
        await Clients.All.SendAsync("UserConnected",
Context.UserIdentifier);
        await base.OnConnectedAsync();
    }

    public override async Task OnDisconnectedAsync(Exception
exception)
    {

```

```

        await Clients.All.SendAsync("UserDisconnected",
Context.UserIdentifier);
        await base.OnDisconnectedAsync(exception);
    }
}

// Program.cs
builder.Services.AddSignalR();

var app = builder.Build();

app.MapHub<ChatHub>("/chathub");

```

Client-side JavaScript:

```

// wwwroot/js/chat.js
const connection = new signalR.HubConnectionBuilder()
    .withUrl("/chathub")
    .build();

// Start connection
connection.start().then(function () {
    console.log("Connected to SignalR hub");
}).catch(function (err) {
    console.error(err.toString());
});

// Receive messages
connection.on("ReceiveMessage", function (user, message) {
    const messageElement = document.createElement("div");
    messageElement.textContent = `${user}: ${message}`;

    document.getElementById("messagesList").appendChild(messageElement
);
});

// Send message
document.getElementById("sendButton").addEventListener("click",
function (event) {

```

```

const user = document.getElementById("userInput").value;
const message = document.getElementById("messageInput").value;

connection.invoke("SendMessage", user, message).catch(function
(err) {
    console.error(err.toString());
});

event.preventDefault();
});

```

3.9 Advanced Entity Framework Core

Ở cấp độ Senior, bạn cần nắm vững các tính năng nâng cao của EF Core.

```

// Advanced Configurations
public class ApplicationDbContext : DbContext
{
    protected override void OnModelCreating(ModelBuilder
modelBuilder)
    {
        // Global Query Filters
        modelBuilder.Entity<Product>().HasQueryFilter(p =>
!p.IsDeleted);

        // Value Conversions
        modelBuilder.Entity<Order>()
            .Property(e => e.Status)
            .HasConversion<string>();

        // Owned Types
        modelBuilder.Entity<Customer>().OwnsOne(c => c.Address);

        // Table Splitting
        modelBuilder.Entity<Product>()
            .ToTable("Products")
            .SplitToTable("ProductDetails", tableBuilder =>
            {
                tableBuilder.Property(p => p.Description);
                tableBuilder.Property(p => p.Specifications);
            });
    }
}

```

```

        });

        // Temporal Tables (SQL Server)
        modelBuilder.Entity<Product>().ToTable(tb =>
        tb.IsTemporal());
    }
}

// Raw SQL và Stored Procedures
public async Task<List<Product>> GetProductsByCategoryAsync(int
categoryId)
{
    return await _context.Products
        .FromSqlRaw("EXEC GetProductsByCategory @CategoryId = {0}",
categoryId)
        .ToListAsync();
}

// Bulk Operations
public async Task BulkUpdatePricesAsync(decimal multiplier)
{
    await _context.Database.ExecuteSqlRawAsync(
        "UPDATE Products SET Price = Price * {0}", multiplier);
}

```

Với những kiến thức Senior Level này, bạn đã có thể xây dựng các ứng dụng ASP.NET Core phức tạp, có khả năng mở rộng và bảo mật cao. Tiếp theo, chúng ta sẽ tìm hiểu về Principal Level - nơi tập trung vào kiến trúc hệ thống và chiến lược công nghệ.

4. Principal Level: Kiến trúc hệ thống và chiến lược

Ở cấp độ Principal, bạn không chỉ là một nhà phát triển mà còn là một kiến trúc sư hệ thống và người đưa ra quyết định công nghệ. Bạn cần hiểu sâu về các mô hình kiến trúc phức tạp, có khả năng thiết kế hệ thống có thể mở rộng và duy trì được trong dài hạn.

4.1 Distributed Systems Patterns

Trong các hệ thống phân tán, việc áp dụng các design patterns phù hợp là rất quan trọng để đảm bảo tính nhất quán, độ tin cậy và khả năng mở rộng.

4.1.1 Event-Driven Architecture

Event-Driven Architecture (EDA) là một mô hình kiến trúc trong đó các thành phần của hệ thống giao tiếp thông qua events.

```
// Domain Events
public abstract class DomainEvent
{
    public DateTime OccurredOn { get; protected set; }
    public Guid EventId { get; protected set; }

    protected DomainEvent()
    {
        EventId = Guid.NewGuid();
        OccurredOn = DateTime.UtcNow;
    }
}

public class OrderCreatedEvent : DomainEvent
{
    public int OrderId { get; }
    public int CustomerId { get; }
    public decimal TotalAmount { get; }
    public List<OrderItem> Items { get; }

    public OrderCreatedEvent(int orderId, int customerId, decimal
totalAmount, List<OrderItem> items)
    {
        OrderId = orderId;
        CustomerId = customerId;
        TotalAmount = totalAmount;
        Items = items;
    }
}
```

```

// Event Bus Interface
public interface IEventBus
{
    Task PublishAsync<T>(T @event) where T : DomainEvent;
    void Subscribe<T>(Func<T, Task> handler) where T : DomainEvent;
}

// Event Bus Implementation với RabbitMQ
public class RabbitMQEventBus : IEventBus
{
    private readonly IConnection _connection;
    private readonly IModel _channel;
    private readonly ILogger<RabbitMQEventBus> _logger;

    public RabbitMQEventBus(IConnection connection,
        ILogger<RabbitMQEventBus> logger)
    {
        _connection = connection;
        _channel = _connection.CreateModel();
        _logger = logger;
    }

    public async Task PublishAsync<T>(T @event) where T :
        DomainEvent
    {
        var eventName = typeof(T).Name;
        var message = JsonSerializer.Serialize(@event);
        var body = Encoding.UTF8.GetBytes(message);

        _channel.ExchangeDeclare(exchange: "events", type:
            ExchangeType.Topic);

        _channel.BasicPublish(
            exchange: "events",
            routingKey: eventName,
            basicProperties: null,
            body: body);

        _logger.LogInformation("Published event {EventName} with ID
            {EventId}", eventName, @event.EventId);
    }
}

```



```

    }

    public void Subscribe<T>(Func<T, Task> handler) where T :
DomainEvent
    {
        var eventName = typeof(T).Name;
        var queueName = $"{{eventName}}_queue";

        _channel.ExchangeDeclare(exchange: "events", type:
ExchangeType.Topic);
        _channel.QueueDeclare(queue: queueName, durable: true,
exclusive: false, autoDelete: false);
        _channel.QueueBind(queue: queueName, exchange: "events",
routingKey: eventName);

        var consumer = new EventingBasicConsumer(_channel);
        consumer.Received += async (model, ea) =>
        {
            try
            {
                var body = ea.Body.ToArray();
                var message = Encoding.UTF8.GetString(body);
                var @event = JsonSerializer.Deserialize<T>
(message);

                await handler(@event);
                _channel.BasicAck(deliveryTag: ea.DeliveryTag,
multiple: false);
            }
            catch (Exception ex)
            {
                _logger.LogError(ex, "Error processing event
{{eventName}}", eventName);
                _channel.BasicNack(deliveryTag: ea.DeliveryTag,
multiple: false, requeue: true);
            }
        };

        _channel.BasicConsume(queue: queueName, autoAck: false,
consumer: consumer);
    }

```

```
}  
}
```

4.1.2 Saga Pattern

Saga Pattern quản lý transactions phân tán bằng cách chia chúng thành một chuỗi các local transactions.

```
// Saga State  
public enum OrderSagaState  
{  
    Started,  
    PaymentProcessed,  
    InventoryReserved,  
    OrderConfirmed,  
    Failed,  
    Compensated  
}  
  
// Saga Data  
public class OrderSagaData  
{  
    public Guid SagaId { get; set; }  
    public int OrderId { get; set; }  
    public int CustomerId { get; set; }  
    public decimal Amount { get; set; }  
    public OrderSagaState State { get; set; }  
    public List<string> CompensationActions { get; set; } = new();  
    public DateTime CreatedAt { get; set; }  
    public DateTime UpdatedAt { get; set; }  
}  
  
// Saga Orchestrator  
public class OrderSaga  
{  
    private readonly IEventBus _eventBus;  
    private readonly IPaymentService _paymentService;  
    private readonly IInventoryService _inventoryService;  
    private readonly ISagaRepository _sagaRepository;
```

```

private readonly ILogger<OrderSaga> _logger;

public OrderSaga(
    IEventBus eventBus,
    IPaymentService paymentService,
    IInventoryService inventoryService,
    ISagaRepository sagaRepository,
    ILogger<OrderSaga> logger)
{
    _eventBus = eventBus;
    _paymentService = paymentService;
    _inventoryService = inventoryService;
    _sagaRepository = sagaRepository;
    _logger = logger;
}

public async Task StartSaga(OrderCreatedEvent orderCreated)
{
    var sagaData = new OrderSagaData
    {
        SagaId = Guid.NewGuid(),
        OrderId = orderCreated.OrderId,
        CustomerId = orderCreated.CustomerId,
        Amount = orderCreated.TotalAmount,
        State = OrderSagaState.Started,
        CreatedAt = DateTime.UtcNow,
        UpdatedAt = DateTime.UtcNow
    };

    await _sagaRepository.SaveAsync(sagaData);

    try
    {
        // Step 1: Process Payment
        var paymentResult = await
        _paymentService.ProcessPaymentAsync(
            sagaData.CustomerId, sagaData.Amount);

        if (paymentResult.IsSuccess)
        {

```

```

        sagaData.State = OrderSagaState.PaymentProcessed;
        sagaData.CompensationActions.Add("RefundPayment");
        await _sagaRepository.UpdateAsync(sagaData);

        // Step 2: Reserve Inventory
        var inventoryResult = await
            _inventoryService.ReserveItemsAsync(orderCreated.Items);

        if (inventoryResult.IsSuccess)
        {
            sagaData.State =
                OrderSagaState.InventoryReserved;

            sagaData.CompensationActions.Add("ReleaseInventory");
            await _sagaRepository.UpdateAsync(sagaData);

            // Step 3: Confirm Order
            sagaData.State = OrderSagaState.OrderConfirmed;
            await _sagaRepository.UpdateAsync(sagaData);

            await _eventBus.PublishAsync(new
                OrderConfirmedEvent(sagaData.OrderId));
            _logger.LogInformation("Order saga completed
                successfully for order {OrderId}", sagaData.OrderId);
        }
        else
        {
            await CompensateSaga(sagaData, "Inventory
                reservation failed");
        }
    }
    else
    {
        await CompensateSaga(sagaData, "Payment processing
            failed");
    }
}
catch (Exception ex)
{

```

```

        _logger.LogError(ex, "Error in order saga for order
{OrderId}", sagaData.OrderId);
        await CompensateSaga(sagaData, ex.Message);
    }
}

private async Task CompensateSaga(OrderSagaData sagaData,
string reason)
{
    _logger.LogWarning("Compensating saga {SagaId} for order
{OrderId}. Reason: {Reason}",
        sagaData.SagaId, sagaData.OrderId, reason);

    sagaData.State = OrderSagaState.Failed;
    await _sagaRepository.UpdateAsync(sagaData);

    // Execute compensation actions in reverse order
    for (int i = sagaData.CompensationActions.Count - 1; i >=
0; i--)
    {
        var action = sagaData.CompensationActions[i];
        try
        {
            switch (action)
            {
                case "RefundPayment":
                    await
_paymentService.RefundPaymentAsync(sagaData.CustomerId,
sagaData.Amount);
                    break;
                case "ReleaseInventory":
                    await
_inventoryService.ReleaseReservationAsync(sagaData.OrderId);
                    break;
            }
        }
        catch (Exception ex)
        {
            _logger.LogError(ex, "Failed to execute
compensation action {Action} for saga {SagaId}",

```

```

        action, sagaData.SagaId);
    }
}

sagaData.State = OrderSagaState.Compensated;
await _sagaRepository.UpdateAsync(sagaData);

await _eventBus.PublishAsync(new
OrderFailedEvent(sagaData.OrderId, reason));
}
}

```

4.1.3 Idempotency

Idempotency đảm bảo rằng thực hiện cùng một operation nhiều lần sẽ có kết quả giống như thực hiện một lần.

```

// Idempotency Key Attribute
public class IdempotentAttribute : ActionFilterAttribute
{
    public override async Task
    OnActionExecutionAsync(ActionExecutingContext context,
    ActionExecutionDelegate next)
    {
        var idempotencyKey =
        context.HttpContext.Request.Headers["Idempotency-
        Key"].FirstOrDefault();

        if (string.IsNullOrEmpty(idempotencyKey))
        {
            context.Result = new
            BadRequestObjectResult("Idempotency-Key header is required");
            return;
        }

        var cacheService =
        context.HttpContext.RequestServices.GetRequiredService<IDistributed
        Cache>();
        var cacheKey = $"idempotency:{idempotencyKey}";
    }
}

```

```

        // Check if request was already processed
        var cachedResult = await
cacheService.GetStringAsync(cacheKey);
        if (cachedResult != null)
        {
            var result =
JsonSerializer.Deserialize<IdempotentResult>(cachedResult);
            context.Result = new ObjectResult(result.Data) {
StatusCode = result.StatusCode };
            return;
        }

        // Execute the action
        var executedContext = await next();

        // Cache the result
        if (executedContext.Result is ObjectResult objectResult)
        {
            var idempotentResult = new IdempotentResult
            {
                StatusCode = objectResult.StatusCode ?? 200,
                Data = objectResult.Value
            };

            var serializedResult =
JsonSerializer.Serialize(idempotentResult);
            var options = new DistributedCacheEntryOptions
            {
                AbsoluteExpirationRelativeToNow =
TimeSpan.FromHours(24)
            };

            await cacheService.SetStringAsync(cacheKey,
serializedResult, options);
        }
    }
}

public class IdempotentResult

```

```

{
    public int StatusCode { get; set; }
    public object Data { get; set; }
}

// Usage
[HttpPost]
[Idempotent]
public async Task<IActionResult> CreateOrder([FromBody]
CreateOrderRequest request)
{
    // Order creation logic
    var order = await _orderService.CreateOrderAsync(request);
    return CreatedAtAction(nameof(GetOrder), new { id = order.Id },
order);
}

```

Những patterns này giúp xây dựng các hệ thống phân tán mạnh mẽ, có khả năng chịu lỗi và đảm bảo tính nhất quán dữ liệu trong môi trường phức tạp.

4.2 Advanced Performance Tuning and Scalability

Ở cấp độ Principal, bạn cần hiểu sâu về performance tuning và có khả năng thiết kế hệ thống có thể scale từ hàng nghìn đến hàng triệu người dùng.

4.2.1 Advanced Caching Strategies

```

// Multi-level Caching Strategy
public class MultiLevelCacheService
{
    private readonly IMemoryCache _l1Cache;
    private readonly IDistributedCache _l2Cache;
    private readonly ILogger<MultiLevelCacheService> _logger;

    public async Task<T> GetAsync<T>(string key, Func<Task<T>>
factory, TimeSpan? expiration = null)
    {
        // L1 Cache (Memory)

```



```

        if (_l1Cache.TryGetValue(key, out T cachedValue))
        {
            return cachedValue;
        }

        // L2 Cache (Distributed)
        var distributedValue = await _l2Cache.GetStringAsync(key);
        if (distributedValue != null)
        {
            var deserializedValue = JsonSerializer.Deserialize<T>(distributedValue);
            _l1Cache.Set(key, deserializedValue,
                TimeSpan.FromMinutes(5));
            return deserializedValue;
        }

        // Factory method
        var value = await factory();
        if (value != null)
        {
            // Set in both caches
            _l1Cache.Set(key, value, expiration ??
                TimeSpan.FromMinutes(5));

            var serializedValue = JsonSerializer.Serialize(value);
            await _l2Cache.SetStringAsync(key, serializedValue, new
                DistributedCacheEntryOptions
                {
                    AbsoluteExpirationRelativeToNow = expiration ??
                        TimeSpan.FromHours(1)
                });
        }

        return value;
    }
}

```

4.2.2 Database Optimization

```
// Connection Pooling và Query Optimization
public class OptimizedProductRepository
{
    private readonly ApplicationDbContext _context;

    // Compiled Queries cho performance tốt hơn
    private static readonly Func<ApplicationDbContext, int,
Task<Product>> GetProductByIdQuery =
    EF.CompileAsyncQuery((ApplicationDbContext context, int id)
=>
        context.Products.FirstOrDefault(p => p.Id == id));

    public async Task<Product> GetProductByIdAsync(int id)
    {
        return await GetProductByIdQuery(_context, id);
    }

    // Batch Operations
    public async Task BulkUpdatePricesAsync(Dictionary<int,
decimal> priceUpdates)
    {
        var productIds = priceUpdates.Keys.ToList();
        var products = await _context.Products
            .Where(p => productIds.Contains(p.Id))
            .ToListAsync();

        foreach (var product in products)
        {
            if (priceUpdates.TryGetValue(product.Id, out var
newPrice))
            {
                product.Price = newPrice;
            }
        }

        await _context.SaveChangesAsync();
    }
}
```

```

// Read-only queries với NoTracking
public async Task<IEnumerable<ProductSummary>>
GetProductSummariesAsync()
{
    return await _context.Products
        .AsNoTracking()
        .Select(p => new ProductSummary
        {
            Id = p.Id,
            Name = p.Name,
            Price = p.Price
        })
        .ToListAsync();
}
}

```

4.3 Domain-Driven Design (DDD) và Clean Architecture

DDD và Clean Architecture giúp tổ chức code theo business domain và tạo ra các hệ thống dễ bảo trì, kiểm thử.

```

// Domain Layer - Entities
public class Order : AggregateRoot
{
    private readonly List<OrderItem> _items = new();

    public int Id { get; private set; }
    public CustomerId CustomerId { get; private set; }
    public OrderStatus Status { get; private set; }
    public Money TotalAmount { get; private set; }
    public DateTime CreatedAt { get; private set; }

    public IReadOnlyList<OrderItem> Items => _items.AsReadOnly();

    private Order() { } // EF Constructor

    public static Order Create(CustomerId customerId,
List<OrderItem> items)
    {
        var order = new Order

```

```

        {
            CustomerId = customerId,
            Status = OrderStatus.Pending,
            CreatedAt = DateTime.UtcNow
        };

        foreach (var item in items)
        {
            order.AddItem(item);
        }

        order.AddDomainEvent(new OrderCreatedEvent(order.Id,
customerId.Value, order.TotalAmount.Amount, items));
        return order;
    }

    public void AddItem(OrderItem item)
    {
        if (Status != OrderStatus.Pending)
            throw new DomainException("Cannot add items to a non-
pending order");

        _items.Add(item);
        RecalculateTotal();
    }

    private void RecalculateTotal()
    {
        TotalAmount = new Money(_items.Sum(i => i.Price.Amount *
i.Quantity));
    }
}

// Value Objects
public class Money : ValueObject
{
    public decimal Amount { get; }
    public string Currency { get; }

    public Money(decimal amount, string currency = "USD")

```

```

    {
        if (amount < 0)
            throw new ArgumentException("Amount cannot be
negative");

        Amount = amount;
        Currency = currency;
    }

    protected override IEnumerable<object> GetEqualityComponents()
    {
        yield return Amount;
        yield return Currency;
    }
}

// Application Layer - Use Cases
public class CreateOrderUseCase
{
    private readonly IOrderRepository _orderRepository;
    private readonly IProductRepository _productRepository;
    private readonly IUnitOfWork _unitOfWork;
    private readonly IDomainEventDispatcher _eventDispatcher;

    public async Task<OrderDto> ExecuteAsync(CreateOrderCommand
command)
    {
        // Validate products exist and have sufficient stock
        var products = await
_productRepository.GetByIdsAsync(command.ProductIds);

        var orderItems = command.Items.Select(item =>
        {
            var product = products.First(p => p.Id ==
item.ProductId);
            return OrderItem.Create(product.Id, item.Quantity,
product.Price);
        }).ToList();

        // Create order

```

```

        var order = Order.Create(new
CustomerId(command.CustomerId), orderItems);

        await _orderRepository.AddAsync(order);
        await _unitOfWork.SaveChangesAsync();

        // Dispatch domain events
        await
_eventDispatcher.DispatchEventsAsync(order.DomainEvents);

        return OrderDto.FromEntity(order);
    }
}

// Infrastructure Layer - Repository Implementation
public class OrderRepository : IOrderRepository
{
    private readonly ApplicationDbContext _context;

    public async Task<Order> GetByIdAsync(int id)
    {
        return await _context.Orders
            .Include(o => o.Items)
            .FirstOrDefaultAsync(o => o.Id == id);
    }

    public async Task AddAsync(Order order)
    {
        await _context.Orders.AddAsync(order);
    }
}

```

4.4 Observability (Distributed Tracing, Metrics, Logging)

Observability là khả năng hiểu được trạng thái nội bộ của hệ thống thông qua các outputs mà nó tạo ra.

```

// Distributed Tracing với OpenTelemetry
public class Startup
{

```

```

public void ConfigureServices(IServiceCollection services)
{
    services.AddOpenTelemetryTracing(builder =>
    {
        builder
            .AddAspNetCoreInstrumentation()
            .AddHttpClientInstrumentation()
            .AddEntityFrameworkCoreInstrumentation()
            .AddJaegerExporter(options =>
            {
                options.AgentHost = "localhost";
                options.AgentPort = 6831;
            });
    });

    // Custom metrics
    services.AddOpenTelemetryMetrics(builder =>
    {
        builder
            .AddAspNetCoreInstrumentation()
            .AddPrometheusExporter();
    });
}

// Custom Metrics
public class OrderMetrics
{
    private readonly Counter<long> _ordersCreated;
    private readonly Histogram<double> _orderProcessingTime;
    private readonly Gauge<int> _activeOrders;

    public OrderMetrics(IMeterFactory meterFactory)
    {
        var meter = meterFactory.Create("OrderService");

        _ordersCreated = meter.CreateCounter<long>
("orders_created_total", "Total number of orders created");
        _orderProcessingTime = meter.CreateHistogram<double>
("order_processing_duration_seconds", "Order processing time");
    }
}

```

```

        _activeOrders = meter.CreateGauge<int>("active_orders",
"Number of active orders");
    }

    public void IncrementOrdersCreated() => _ordersCreated.Add(1);
    public void RecordOrderProcessingTime(double duration) =>
_orderProcessingTime.Record(duration);
    public void SetActiveOrders(int count) =>
_activeOrders.Record(count);
}

// Structured Logging
public class OrderService
{
    private readonly ILogger<OrderService> _logger;
    private readonly OrderMetrics _metrics;

    public async Task<Order> CreateOrderAsync(CreateOrderRequest
request)
    {
        using var activity = Activity.StartActivity("CreateOrder");
        activity?.SetTag("customer.id",
request.CustomerId.ToString());
        activity?.SetTag("order.items.count",
request.Items.Count.ToString());

        var stopwatch = Stopwatch.StartNew();

        try
        {
            _logger.LogInformation("Creating order for customer
{CustomerId} with {ItemCount} items",
request.CustomerId, request.Items.Count);

            var order = await ProcessOrderCreation(request);

            stopwatch.Stop();

            _metrics.RecordOrderProcessingTime(stopwatch.Elapsed.TotalSeconds)
;

```



```

        _metrics.IncrementOrdersCreated();

        _logger.LogInformation("Successfully created order
{OrderId} for customer {CustomerId}",
            order.Id, request.CustomerId);

        activity?.SetTag("order.id", order.Id.ToString());
        activity?.SetStatus(ActivityStatusCode.Ok);

        return order;
    }
    catch (Exception ex)
    {
        _logger.LogError(ex, "Failed to create order for
customer {CustomerId}", request.CustomerId);
        activity?.SetStatus(ActivityStatusCode.Error,
ex.Message);
        throw;
    }
}
}

```

4.5 Advanced Security Topics

```

// Security Headers Middleware
public class SecurityHeadersMiddleware
{
    public async Task InvokeAsync(HttpContext context,
RequestDelegate next)
    {
        // Content Security Policy
        context.Response.Headers.Add("Content-Security-Policy",
            "default-src 'self'; script-src 'self' 'unsafe-inline';
style-src 'self' 'unsafe-inline'");

        // Prevent clickjacking
        context.Response.Headers.Add("X-Frame-Options", "DENY");

        // XSS Protection
    }
}

```

```

        context.Response.Headers.Add("X-XSS-Protection", "1;
mode=block");

        // MIME type sniffing protection
        context.Response.Headers.Add("X-Content-Type-Options",
"nosniff");

        // Referrer Policy
        context.Response.Headers.Add("Referrer-Policy", "strict-
origin-when-cross-origin");

        // HSTS
        if (context.Request.IsHttps)
        {
            context.Response.Headers.Add("Strict-Transport-
Security",
                "max-age=31536000; includeSubDomains; preload");
        }

        await next(context);
    }
}

// Rate Limiting
public class RateLimitingMiddleware
{
    private readonly IMemoryCache _cache;
    private readonly RequestDelegate _next;

    public async Task InvokeAsync(HttpContext context)
    {
        var clientId = GetClientIdentifier(context);
        var key = $"rate_limit_{clientId}";

        if (_cache.TryGetValue(key, out int requestCount))
        {
            if (requestCount >= 100) // 100 requests per minute
            {
                context.Response.StatusCode = 429;
            }
        }
    }
}

```

```

        await context.Response.WriteAsync("Rate limit
exceeded");

        return;
    }

    _cache.Set(key, requestCount + 1,
    TimeSpan.FromMinutes(1));
}
else
{
    _cache.Set(key, 1, TimeSpan.FromMinutes(1));
}

    await _next(context);
}
}

```

4.6 Advanced Deployment and DevOps

```

# Azure DevOps Pipeline
trigger:
- main

variables:
    buildConfiguration: 'Release'
    dockerRegistryServiceConnection: 'myregistry'
    imageRepository: 'mywebapp'
    containerRegistry: 'myregistry.azurecr.io'
    dockerfilePath: '$(Build.SourcesDirectory)/Dockerfile'
    tag: '$(Build.BuildId)'

stages:
- stage: Build
    displayName: Build and Test
    jobs:
    - job: Build
        displayName: Build
        pool:
            vmImage: 'ubuntu-latest'
        steps:

```

```

- task: DotNetCoreCLI@2
  displayName: Restore
  inputs:
    command: 'restore'
    projects: '**/*.csproj'

- task: DotNetCoreCLI@2
  displayName: Build
  inputs:
    command: 'build'
    projects: '**/*.csproj'
    arguments: '--configuration $(buildConfiguration)'

- task: DotNetCoreCLI@2
  displayName: Test
  inputs:
    command: 'test'
    projects: '**/*Tests/*.csproj'
    arguments: '--configuration $(buildConfiguration) --collect
"Code coverage"'

- task: Docker@2
  displayName: Build and push Docker image
  inputs:
    command: buildAndPush
    repository: $(imageRepository)
    dockerfile: $(dockerfilePath)
    containerRegistry: $(dockerRegistryServiceConnection)
    tags: |
      $(tag)
      latest

- stage: Deploy
  displayName: Deploy to Production
  dependsOn: Build
  condition: succeeded()
  jobs:
    - deployment: Deploy
      displayName: Deploy
      pool:

```

```
    vmImage: 'ubuntu-latest'
    environment: 'production'
    strategy:
      runOnce:
        deploy:
          steps:
            - task: AzureWebAppContainer@1
              displayName: 'Deploy to Azure Web App'
              inputs:
                azureSubscription: 'Azure-Connection'
                appName: 'mywebapp-prod'
                containers:
                  '$(containerRegistry)/$(imageRepository):$(tag)'
```

Kết luận

Tài liệu này đã cung cấp cho bạn một hành trình học tập toàn diện về ASP.NET Core, từ những khái niệm cơ bản nhất cho đến các kỹ thuật nâng cao nhất. Bằng cách theo dõi cấu trúc từ Junior đến Principal Level, bạn đã được trang bị:

Junior Level: Nền tảng vững chắc với C#, ASP.NET Core basics, MVC, Razor Pages, và các khái niệm cốt lõi.

Middle Level: Kỹ năng phát triển ứng dụng thực tế với Entity Framework Core, CRUD operations, validation, authentication, API development, logging, và unit testing.

Senior Level: Chuyên môn sâu về performance optimization, security, advanced authentication, microservices, containerization, deployment strategies, integration testing, real-time communication, và advanced EF Core.

Principal Level: Tâm nhìn kiến trúc với distributed systems patterns, advanced performance tuning, Domain-Driven Design, Clean Architecture, observability, advanced security, và DevOps practices.

Để thành thạo ASP.NET Core, hãy:

1. **Thực hành thường xuyên:** Xây dựng các dự án thực tế, từ đơn giản đến phức tạp.
2. **Đọc source code:** Nghiên cứu source code của ASP.NET Core và các thư viện mã nguồn mở.

3. **Tham gia cộng đồng:** Đóng góp vào các dự án mã nguồn mở, tham gia các diễn đàn và hội thảo.
4. **Cập nhật kiến thức:** Theo dõi các bản cập nhật mới của .NET và ASP.NET Core.
5. **Áp dụng best practices:** Luôn tuân thủ các nguyên tắc SOLID, clean code, và security best practices.

Với kiến thức từ tài liệu này và sự thực hành liên tục, bạn sẽ có thể xây dựng các ứng dụng web hiện đại, có khả năng mở rộng và bảo mật cao với ASP.NET Core.

Tác giả: Manus AI

Ngày hoàn thành: 2025

Phiên bản: 1.0

References

- [1] Microsoft Learn - ASP.NET Core Overview: <https://learn.microsoft.com/en-us/aspnet/core/overview>
- [2] Microsoft Learn - ASP.NET Core Fundamentals: <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/>
- [3] .NET Download Page: <https://dotnet.microsoft.com/en-us/download/dotnet>
- [4] Entity Framework Core Documentation: <https://learn.microsoft.com/en-us/ef/core/>
- [5] ASP.NET Core Security Documentation: <https://learn.microsoft.com/en-us/aspnet/core/security/>
- [6] Docker Documentation: <https://docs.docker.com/>
- [7] Azure App Service Documentation: <https://learn.microsoft.com/en-us/azure/app-service/>
- [8] AWS Documentation: <https://docs.aws.amazon.com/>
- [9] SignalR Documentation: <https://learn.microsoft.com/en-us/aspnet/core/signalr/>
- [10] OpenTelemetry .NET Documentation: <https://opentelemetry.io/docs/instrumentation/net/>